

Behzad Moghadam

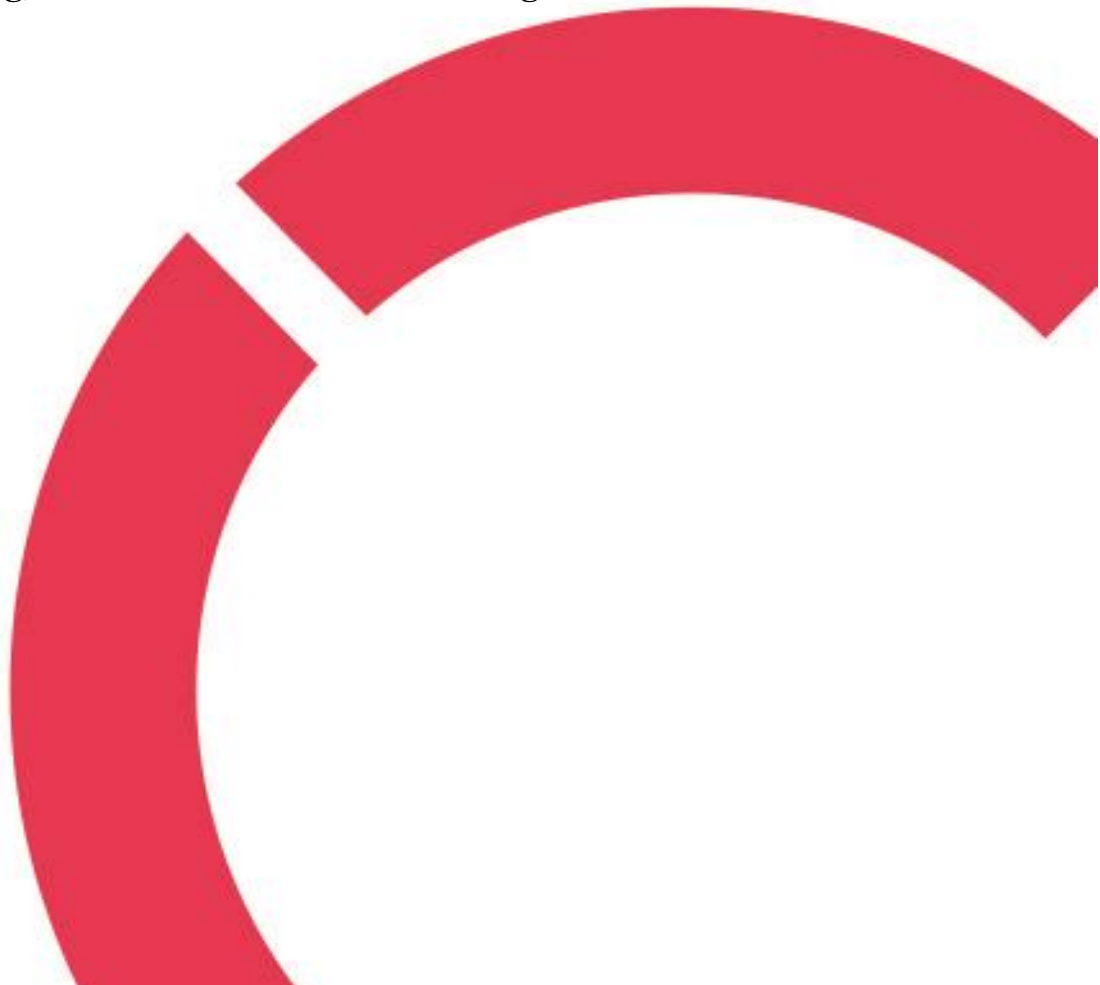
**CYCLE TIME PREDICTION AND SHORT
TERM BOTTLENECK FORECASTING IN AN ASSEMBLY LINE
USING TIME SERIES MACHINE LEARNING WITH DRIFT
DETECTION**

Thesis

CENTRIA UNIVERSITY OF APPLIED SCIENCES

Master of Engineering, Cloud-Driven Artificial Intelligence

April 2026



ABSTRACT

Centria University of Applied Sciences	Date April 2026	Author Behzad Moghadam
Degree programme Master of engineering, cloud driven artificial intelligence		
Name of thesis CYCLE TIME PREDICTION AND SHORT TERM BOTTLENECK FORECASTING IN AN ASSEMBLY LINE USING TIME SERIES MACHINE LEARNING WITH DRIFT DETECTION		
Centria supervisor Dr. Aliasghar Khavasi and Dr. Tomi Pitkäaho		Pages 80
This thesis was carried out in an industrial assembly line environment. The goal was to develop and evaluate drift aware machine learning models. The models addressed two tasks including cycle time prediction and bottleneck forecasting. Both tasks were tested under changing manufacturing conditions. The theoretical framework covered three areas. The first was time series forecasting in manufacturing. The second was concept drift and adaptive learning. The third was bottleneck detection and forecasting. A manufacturing dataset was pre-processed and organised into sequential input windows. Drift was introduced through controlled simulation. Three phases were defined as stable, drift and recovery. For cycle time prediction, three models were compared. These were ARIMA, standalone LSTM and drift aware LSTM with adaptive windowing. For bottleneck forecasting, static and adaptive LSTM models were evaluated. Three forecast horizons were used. Performance was measured using precision, F1 score, balanced accuracy and lead time. The drift aware model performed best in the recovery phase. The stable phase showed no advantage from adaptation and the drift phase produced mixed results. Both bottleneck models provided positive lead time across all horizons. The adaptive model showed higher precision, it produced fewer false positive predictions. In contrast, the static model demonstrated better overall detection performance when considering combined metrics. The study concluded that drift aware machine learning can support short term prediction. It is an early warning in dynamic assembly line environments. In practical values, it depends on three factors including detection accuracy, alert reliability and implementation constraints.		

Key words adaptive learning, assembly line, bottleneck forecasting, concept drift, cycle time prediction, drift detection, LSTM, non-stationary processes, phase dependent thresholds, time series forecasting
--

CONCEPT DEFINITIONS

ADWIN

Adaptive Windowing

ARIMA

Autoregressive Integrated Moving Average

Bottleneck

A machine, workstation or process stage that restricts overall production flow and reduces throughput.

Concept Drift

A shift over time in the relationship between input feature and the target outcome caused by changing operating conditions.

Cycle Time

The duration needed to complete a manufacturing operation or work on a component.

LSTM

Long Short term Memory

WIP

Work in Progress

ABSTRACT
CONCEPT DEFINITIONS
CONTENTS

1 INTRODUCTION.....	7
2 BACKGROUND AND LITERATURE REVIEW	9
2.1 Industrial process overview.....	9
2.1.1 Data flows	9
2.1.2 Assembly line operating states	11
2.2 Time series forecasting in manufacturing	13
2.3 Concept drift and adaptive learning	13
2.4 Bottleneck detection, forecasting and monitoring	14
2.5 Existing control and optimisation methods	14
2.5.1 Adaptive and industrial optimisation	15
2.6 Success criteria and key performance indicators (KPIs)	16
2.7 State estimation	19
3 IMPLEMENTATION, INTEGRATION	21
3.1 Requirements definition	22
3.2 Dataset definition	27
3.3 Data acquisition and processing pipeline.....	28
3.3.1 Data loading	29
3.3.2 Data cleaning	29
3.3.3 Feature extraction	30
3.3.4 Sequence generation	32
3.3.5 Definition of stable, drift and recovery phases	33
3.4 Baseline, model implementation	35
3.5 Adaptive intelligence layer	35
3.6 Model evaluation and KPI reporting	38
3.7 Practical challenges and benefits	40
3.7.1 Challenges.....	41
3.7.2 Benefits.....	42
4 EVALUATION, RESULTS AND REFINEMENT	43
4.1 Experimental design	44
4.1.1 Evaluation phases	44
4.1.2 Scenario definition	45
4.1.3 Data and sequence setup	45
4.2 Results for cycle time prediction.....	46
4.2.1 Performance in the stable phase.....	47
4.2.2 Performance in the drift phase	47
4.2.3 Performance in the recovery phase.....	48
4.2.4 Comparative results for ARIMA, standalone LSTM and adaptive LSTM ...	49

4.3 Results for bottleneck forecasting	50
4.3.1 Bottleneck class behaviour across phases	50
4.3.2 Performance at forecast horizons $t+1$, $t+3$ and $t+5$	51
4.3.3 Comparative Results for Static and Adaptive LSTM	54
4.3.4 Lead Time analysis	56
4.4 Drift aware behaviour and KPI analysis	57
4.4.1 Detection delay and false positives	58
4.4.2 Residual analysis for adaptive cycle time prediction.....	59
4.5 Model refinement and trade offs	61
4.5.1 Refinement of adaptive updating logic	61
4.5.2 Threshold refinement in bottleneck forecasting	62
4.5.3 Accuracy versus stability	63
4.5.4 Responsiveness versus false alarms.....	64
4.6 Practical challenges and benefits revisited	65
4.6.1 Challenges under simulated drift	66
4.6.2 Benefits of adaptive forecasting and early warning	67
4.6.3 Deployment oriented considerations	68
4.7 Limitations and generalizability	69
4.7.1 Dataset and simulation limitations.....	69
4.7.2 Methodological limitations.....	70
4.7.3 Generalizability to similar assembly line settings.....	72
5 DISCUSSION	73
5.1 Interpretation of research question 1	73
5.2 Interpretation of research question 2	75
5.3 Interpretation of research question 3	76
5.4 Discussion summary	78
REFERENCE.....	79
 FIGURES	
FIGURE 1. Data flow (Richard and Belacel, 2024, p. 8)	10
FIGURE 2. Simulation environment	38
FIGURE 3. Precision score in drift phase.....	52
FIGURE 4. F1 score in drift phase	53
FIGURE 5. Balanced accuracy score.....	53
FIGURE 6. Static and adaptive performance	55
FIGURE 7. Average lead time.....	57
FIGURE 8. Delay and positive detection	59
FIGURE 9. Residual in LSTM and ASWIN	60
 TABLE	
TABLE 1. KPI and success criteria (Wang et al, 2021, p. 13).	18
TABLE 2. Functional requirements.....	23
TABLE 3. Nonfunctional requirements.....	25

TABLE 4. Domain specific requirements	26
TABLE 5. Dataset summary	29
TABLE 6. Data cleaning.....	30
TABLE 7. Input feature creation	31
TABLE 8. Sequence generation	32
TABLE 9. Definition of stable, drift and recovery phases	34
TABLE 10. Dataset dividing	44
TABLE 11. ARIMA, LSTM, LSTM+ADWIN compressive	49

1 INTRODUCTION

Manufacturing assembly lines aim to operate with a minimum number of stations while minimising total costs and maximizing production speed (Nourmohammadi, Arbaoui, Fathi & Dolgui 2025, 1.) In ideal feedback loops, some processes are stationary, but practice industrial operating assembly line are completely dynamic. In a non-stationary station, some factors such as tool wear, product mix variation, changing shift patterns, upstream disturbances and intermittent machine faults introduce non stationarity or concept drift has direct effect on process data in production lines. Concept drift occurs when the connection between input and output variables evolves over time, usually as a result of changes in the operating environment (Gomes et al. 2017,3). Also, these factors significantly reduce the reliability of traditional, static forecasting approaches. They led to inaccurate predictions and delayed detection of emerging bottlenecks. This thesis develops and evaluates drift aware time series machine learning models for short term cycle time prediction and bottleneck forecasting in assembly lines. It also identifies the practical requirements for deploying a model in real industrial environments. To achieve this aim, the study uses four concrete steps. First, baseline models consist of Autoregressive Integrated Moving Average (ARIMA) and standalone Long Short term Memory (LSTM). They were implemented to establish performance references under stable conditions. Second, Adaptive Windowing (ADWIN) based drift detection was integrated into the LSTM pipeline which produces an adaptive cycle time predictor. Third, short term bottleneck forecasting was formulated as a binary classification task. Fourth, all models were evaluated across three phases: stable, drift and recovery. Three research question's structure define is defined. The first question asks how integrating ADWIN drift detection into an LSTM pipeline affects the accuracy and robustness of short-term cycle time prediction. It is compared to a standalone LSTM and ARIMA. The second asks to what extent time series machine learning models can forecast short-term bottlenecks in an assembly line operating under dynamic conditions. The third asks what practical requirements, challenges, and benefits arise when deploying drift-aware cycle time and bottleneck forecasting models in a real industrial assembly line environment. ARIMA combines autoregressive and moving average components with a differencing step that handles non stationarity in the input series.

(Alenezy et al. 2022,4) Beyond accuracy, robustness measures how consistently a model maintains acceptable performance when operating conditions fluctuate. In manufacturing systems, this is quantified through the variance of key performance metrics across stable, drift and recovery phases. (Pagone et al. 2023,1). Robustness is assessed through three measures: performance degradation around detected drift events, recovery time after a distribution change and drift detection delay as reported by ADWIN. The sliding window used by ADWIN has no fixed size, it expands or contracts online based on the rate of change in the incoming data stream.(Bifet and Gavaldà 2007,443) Success criteria cover both predictive accuracy and operational relevance. RMSE and MAE measure the deviation between predicted and observed cycle times and enable direct comparison between adaptive and static models across stable and non-stationary conditions.

For the second research question, success criteria focus on the model's ability to detect imminent bottlenecks with enough lead time for operational intervention. The relevant metrics are recalling the proportion of actual bottleneck events, correctly identified precision the proportion of bottleneck alerts that are correct and lead time, which measures how early a warning is issued before the event occurs. Detection consistency across operating phases indicates model stability. Reduction of unplanned downtime captures the practical operational impact (Davis and Goadrich 2006,236). For the third research question, each KPI is linked to a quantitative measure Downtime is recorded as the total duration and frequency of unplanned stoppages within a specific period. Utilisation is the ratio of effective processing time to available production time across workstations. Stability is measured through the variance of prediction errors over time. Bottleneck detection quality is evaluated by recall, precision and lead time. Robustness under concept drift is captured through detection delay, recovery time and the magnitude of performance degradation across phase transitions. The theoretical framework covers time series forecasting, concept drift, adaptive learning and bottleneck methods. The methodology chapter describes the dataset, preprocessing steps, model architectures, simulation design and evaluation metrics. The results chapter presents model performance across all phases. The discussion summary interprets the findings in relation to the three research questions and presents the results.

2 BACKGROUND AND LITERATURE REVIEW

Production systems generate sequential data. Assembly line behaviour depends on recent process history, which makes time series forecasting a natural fit. In assembly lines, variables such as processing time, machine availability and upstream flow conditions work continuously. They directly shape operational performance. Traditional methods including ARIMA, and exponential smoothing are the standard tools for industrial prediction. They both rely on assumptions of statistical stability that break down under nonstationary conditions. Product mix variation, machine disturbances and shift changes are recurring sources of such nonstationary in modern assembly line environments. These limitations have directed research toward adaptive, learning based approaches, specifically LSTM networks. Adaptive learning approaches are often combined with drift detection methods such as ADWIN. This chapter reviews the literature on manufacturing forecasting, concept drift, adaptive learning and bottleneck forecasting.

2.1 Industrial process overview

An industrial process is a sequence of connected production activities that transform inputs into finished outputs. In an assembly line, each workstation performs a specific task. Its overall performance depends on smooth flow between stages. Processing time, machine availability, and upstream delays all affect production efficiency. This flow may change over time, making monitoring, forecasting, and bottleneck identification more challenging. It is essential to maintaining production performance.

2.1.1 Data flows

Stage 0 (Data stream): The framework begins by gathering a data stream consisting of two segments, including an initial set of labelled data points of a defined length. Stage 1 (Initial training): The model is trained on the initial labelled data segment. The initial labelled data

feature points are divided into two subsets. The first subset is used to fill a sliding window for model fitting and is referred to as the model fitting subset. The second subset is used for testing and for calculating the root mean squared error and is referred to as the model testing subset or the buffer. As new labelled data becomes available, the model fitting window is filled first. Once the fitting window is full, the buffer is filled. When both the model fitting window and the buffer are full, a regression model is trained and its initial RMSE is calculated by comparing the predicted values against the actual target values. This error value is stored for use in subsequent drift monitoring as $Z1$. The trained model is then stored for use in predictions or for replacement in later stages and the buffer is cleared (see FIGURE 1)(Richard and Belacel 2024,7).

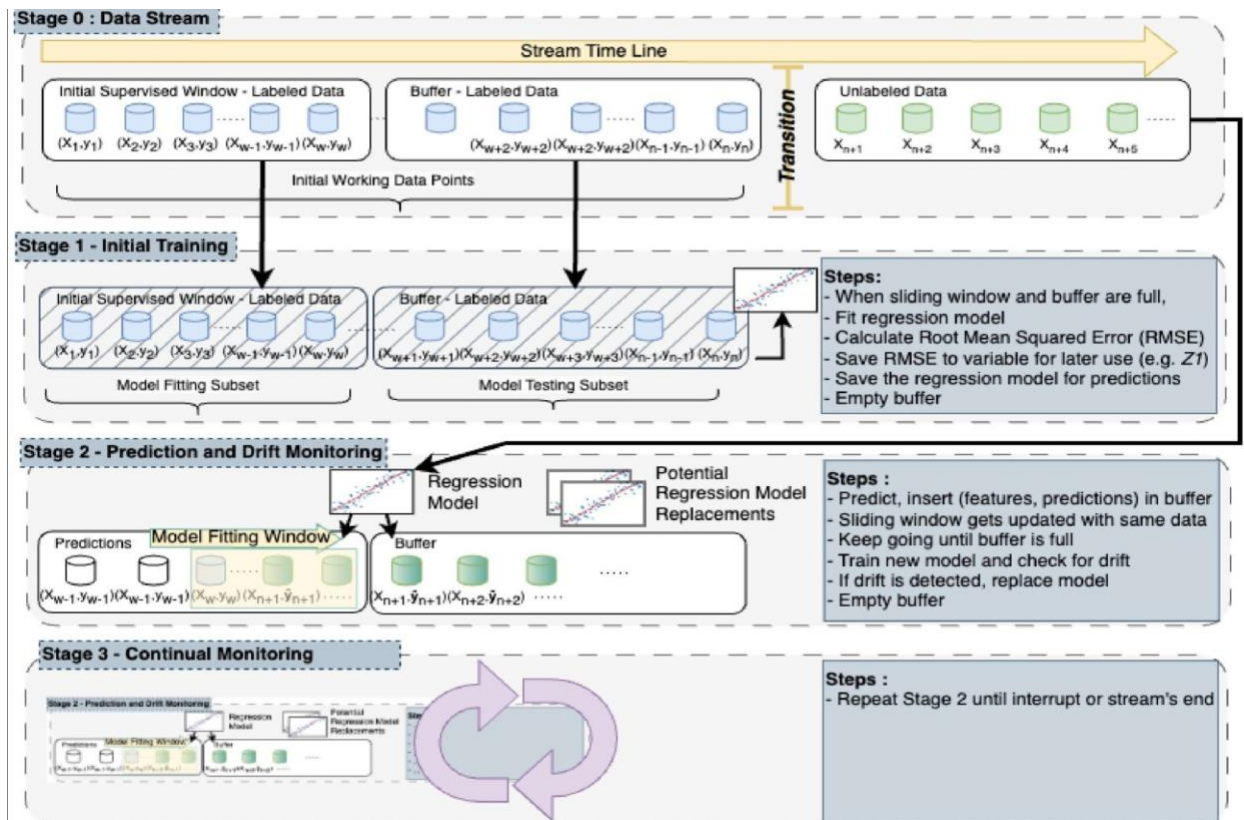


FIGURE 1. Data flow (Richard and Belacel 2024,7).

Stage 2 involves prediction and drift monitoring, where the model predicts the target variable upon the arrival of a new unlabelled sample. The input features of the unlabelled sample are combined with the predicted target value and added to the buffer. At the same time, the model fitting window is updated by adding the new data and removing the oldest entry within the sliding window. When the buffer becomes full, the model is retrained using the data in the model fitting window. The retrained model is then used to predict the target values of the samples. This updated model is saved in the buffer. A new root mean squared error value is calculated by comparing the predicted values. It is against the previously stored true values in the buffer. This value is referred to as Z_2 . The difference between Z_2 and the previously stored value Z_1 is then calculated. If this difference exceeds a predefined threshold, a drift detection event is registered. This indicates that the data distribution has changed, and the current regression model may need to be replaced. If drift is detected, the current model is replaced with updated version. The buffer is cleared regardless of whether drift is detected or not (see FIGURE 1)(Richard and Belacel 2024,7).

Stage 3 (Continual monitoring): As new data becomes available in the data stream; Stage 2 is repeatedly executed. The system continues monitoring the stream for drift and updates the model, the model fit window and the buffer when necessary. To allow more advanced drift detection and model adaptation in streaming regression, the ADWIN detector can optionally be incorporated in Stage 2 of the proposed method. When ADWIN detects drift, the framework checks for additional evidence of drift using RMSE and takes appropriate action. If ADWIN does not detect drift, the algorithm continues operating without performing any model replacement and the buffer is cleared as required. (see FIGURE 1)(Richard and Belacel 2024,8-9).

2.1.2 Assembly line operating states

Assembly line analysis requires the categorisation of machine and resource states into mutually exclusive operational conditions, as these states directly determine production flow, cycle time

behaviour and bottleneck formation. These states provide the operational context necessary for state-aware optimisation. Block is one of the primary disruption states in assembly line operation. When a downstream buffer reaches capacity, a completed part cannot move forward, leaving the upstream machine stuck in a blocked state until space becomes available. This idle period directly disrupts production flow and is one of the more common triggers for bottleneck formation. Once space becomes available, the part is shifted sharply, and the machine can begin processing the next step if one is available. Blocking disrupts production flow, increases cycle time variation and contributes to bottleneck formation. Processing Times, whether fixed or variable across jobs shapes downstream performance outcomes from cycle time to bottleneck risk. When variable, processing times are modelled as random variables. It follows a common probability distribution. Their variation drives the time series patterns. The machine learning models are used to predict short term production performance. Buffer behaviour: it absorbs short term flow imbalances between workstations. Each time a part moves through, upstream capacity is released. The downstream capacity is consumed, a dynamic that propagates across the line. Variations in buffer levels affect machine utilisation and cycle time and can lead to the formation of bottlenecks. Buffer dynamics therefore play a central role in cycle time prediction and bottleneck forecasting.

Unplanned machine failures are among the hardest variability sources to manage, precisely because they strike without warning and halt production until repairs are done sometimes cascading into starvation or blocking at adjacent station. A machine is considered operational when it is functioning, and it is considered working when it is operational and actively processing parts. Operation dependent failures happen only while the machine is running parts. Time dependent failures can occur at any time, regardless of whether the machine is actively processing or idle. In many industrial production systems, most failures are operation dependent. Uptime refers to the period between a repair and the next failure, while downtime corresponds to the time required to repair the machine. Machine failures influence cycle time variability, bottleneck formation and the dynamic behaviour of the assembly line. These operational disruptions generate patterns in production data that affect forecasting performance and considered when developing predictive models for industrial production systems (Dallery and Gershwin 1992,8-10)

2.2 Time series forecasting in manufacturing

Time series forecasting in manufacturing refers to the use of sequential production data features to forecast future process behaviour, such as cycle time or bottleneck formation. Assembly line performance depends on recent machine states upstream conditions and process history. The forecasting methods preserve temporal order. ARIMA provides a statistical baseline for continuous cycle time prediction. The LSTM networks handle sequential data and learn short term dependencies from rolling production windows. ADWIN-based drift detection is added to the pipeline to respond to changes in operating conditions, which introduces deployment requirements: continuous monitoring of incoming data, detection of shifting process behaviour and selective model updating when performance degrades. These components together define the scope of this study, cycle time prediction, bottleneck forecasting, adaptive model behaviour and deployment considerations in dynamic assembly line environments.

2.3 Concept drift and adaptive learning

Concept drift happens when the communication between past production data and future process outcomes changes over time driven in industrial environments by machine wear, reduced availability, disturbances or process reconfiguration. In this study, drift handling is implemented differently in the two forecasting pipelines. In the cycle time prediction pipeline, a custom ADWIN detector monitors recent squared prediction error and signals when the adaptive LSTM should be retrained. In the bottleneck forecasting pipeline, drift handling is not based on ADWIN. Instead, it uses a performance-triggered monitoring rule in which recent F1 score is checked over a short history window and retraining is triggered when performance falls below the defined threshold. In the implemented code, this monitoring rule is applied within the drift interval only and is also subject to a minimum gap of 40 steps between successive retraining events. Alternative drift detection methods in the literature include drift

, early detection methods and Page-Hinkley, but these are not implemented in the provided codes. In both pipelines, adaptation is carried out through recent-window batch retraining rather than incremental updating. These drift aware mechanisms are used to maintain forecasting performance under changing assembly line conditions.

2.4 Bottleneck detection, forecasting and monitoring

A bottleneck is a machine, stage, or operating condition that restricts overall production flow, causing delays, work accumulation and reduced throughput. Bottlenecks may be static where the same stage remains limiting or dynamic, where the limiting stage shifts over time. Bottleneck detection identifies existing constraints, while bottleneck forecasting predicts whether constraints are likely to emerge. Short term forecasting uses recent production data to warn of near future bottleneck across multiple horizons. It led time measuring how far in advance a warning is issued. Among machine learning approaches, LSTM is directly applicable. It learns patterns from sequential production data and uses recent process history. It estimates future bottlenecks. Effective deployment also requires continuous, data driven monitoring of production conditions. They include processing time and machine availability, while separately assessing whether the forecasting model remains valid as operating conditions change.

2.5 Existing control and optimisation methods

Existing control and optimisation approaches to bottleneck and cycle time management in assembly lines typically operate at three levels. At the first level, line supervisors visually monitor each stage and instruct upstream operators. It slows down when queues grow a reactive approach dependent on individual experience. At the second level, rule-based alerts in the MES trigger warnings. It alerts when a queue surpasses a fixed threshold, automating detection but still without predictive capability. At the third level, Statistical Process Control uses control charts to signal when cycle times exceed predefined limits. It does not forecast future

conditions or system-wide bottlenecks. The models developed in this study follow a three-stage progression. ARIMA extends SPC with linear, univariate cycle time forecasting but without adaptation to changing conditions. Standalone LSTM learns nonlinear, multivariate patterns from system states and flows, capturing more complex relationships, though performance degrades when drift occurs. LSTM combined with ADWIN adds automatic drift detection and selective model retraining, maintaining forecasting performance under changing operating conditions. The ADWIN component uses a dynamically adjusted sliding window that responds to both sudden and gradual concept drift (Saravana & Arunalatha 2025, 7). This progression moves from reactive control to predictive capability, from univariate to multivariate learning and from static to adaptive modelling.

2.5.1 Adaptive and industrial optimisation

Traditional optimisation approaches treated assembly line production systems as static frameworks which are fixed behavioural patterns over time. Under real operating conditions, tool wear, shift changes, and unplanned equipment events affect system performance. These assumptions break down and static methods lose effectiveness as the production environment diverges from their initial parameters. Adaptive industrial optimisation is addressed by introducing continuous monitoring and mechanisms for responding to changing conditions. Historically, controllers for complex industrial systems have been grounded in mechanistic modelling, with dynamic control strategies built on mathematical representations of the underlying physical processes.(Tian, Li, Wang & Zhang 2024, 2.) Within this framework, adaptation is driven by online learning and event-triggered model updates. Periodic retraining is a recognised requirement for sustaining forecasting performance under non-stationary conditions with drift signalled. A drift detection event is registered when the difference between the mean values of the two sub-windows becomes large enough to exceed the boundary established by the Hoeffding bound. (Sun et al. 2016,3).

Central to this adaptive structure is the question of whether operational decisions remain valid as production conditions evolve. Two elements addressed are: monitoring time series data to

identify shifts in statistical behaviour and applying drift detection algorithms such as ADWIN that operate under a data stream paradigm and adjust window sizes dynamically. Time series prediction has become a key component of data analysis in industry 4.0 manufacturing applications.(Kashpruk, Piskor-Ignatowicz & Baranowski 2023, 1) Bottlenecks are systemic constraints that arise at any workstation due to blocking, capacity imbalance or equipment failure. In short term forecasting, the aim is to predict bottleneck conditions. They are identified before they fully develop, rather than being identified only after they have already occurred

2.6 Success criteria and key performance indicators (KPIs)

The forecasting runs of the proposed ADWIN + LSTM framework demonstrate its predictive capability. The proposed framework and its corresponding baselines are assessed using three standard evaluation metrics: RMSE, MAE, and MAPE. These metrics are selected predictive accuracy, robustness and interpretability. It specifically addresses the complexities of multivariate and dynamic industrial environments. These are useful in settings where prediction stability and sensitivity to scale are important considerations. (Saravana & Arunalatha 2025,10.) MAE is a performance metric used to measure the difference between predicted and actual values. It represents the average magnitude of prediction errors MAE is calculated as $MAE = (1/n) \times \sum |y_i - \hat{y}_i|$. Where n = number of observations y_i = actual value, $\hat{y}_i$ = predicted value and Σ = sum over all observations. Mean Absolute Percentage Error (MAPE) measures relative prediction error as a percentage to address limitations of MAE and MSE methods. The large data values inflate results, MAPE interpretation thresholds are <10% (highly accurate), 10-20% (good predictor), 20-50% (reasonable predictor)>50% (inaccurate). Root Mean Square Error (RMSE) known as standard error. It is the square root of the mean squared differences between observed and actual values. RMSE gives greater weight to larger errors and making it sensitive. It is sensitive to significant deviations and is useful for assessing measurement precision

Detection Delay measures the number of production parts between the point. The concept drift is introduced and the point at which ADWIN first detects it. False Positives represents the

number of ADWIN drift signals that occur before the actual drift on set. It performs during the stable evaluation phase. A false positive may trigger unnecessary retraining on stable data and can therefore disturb an already well fitted model. A false positive count of zero indicates that ADWIN remained inactive throughout the stable phase. It is only triggered after the simulated drift began, which reflects reliable drift detection on this dataset. MAE measures the average absolute difference between the model predictions and the actual values. Precision measures the proportion of predicted bottleneck alerts that are correct. Predicted probabilities are converted into binary predictions using a threshold of 0.5. Low precision indicates excessive false bottleneck alerts. In an assembly line setting, this can reduce operator trust and trigger unnecessary interventions. Recall measures the proportion of actual bottleneck events correctly identified. Low recall means true bottlenecks are missed. It leads to avoidable delays and reduced responsiveness in assembly line operations. Precision and recall capture complementary aspects of bottleneck forecasting performance. They will be interpreted together. Lead Time measures how early the model issues a bottleneck warning before the actual event occurs. For each confirmed bottleneck step, the code searches backward up to ten steps to find the earliest prior step. The adaptive model predicted a bottleneck probability of at least 0.5. Both mean and median lead time are reported.

A positive lead time indicates that the model provides advance warning before the bottleneck occurs. Residual analysis is used to evaluate the differences between actual and predicted values. For the adaptive LSTM, mean and standard deviation are reported per evaluation phase. A mean close to zero indicates no systematic over- or underprediction. A broadly centered and unstructured residual distribution, as visualised in the plot, suggests that the model captures the main data pattern without strong directional bias. Residual analysis is used only to the adaptive LSTM model, since this is the model used to evaluate drift aware behaviour. F1 Score is the harmonic means of precision and recall, calculated as $F1\ score(y_true, y_pred, zero_division=0)$. It provides a single combined measure of bottleneck detection performance and is particularly informative when class distribution is imbalanced, as is typically the case in bottleneck classification tasks. Balanced Accuracy, accounts for class imbalance in bottleneck classification by averaging the recall obtained for each class. It is calculated as

`balanced_accuracy_score (y_true, y_pred)`. It is more reliable than standard accuracy when bottleneck events are rare compared to non-bottleneck steps (see TABLE 1).

TABLE 1. KPI and success criteria (Wang, Chien & Trappey 2021, 13).

Category	Metric	Purpose
Drift detection	Detection Delay	Time lag between actual drift occurrence and ADWIN detection.
Drift detection	False Positives	Evaluates whether the drift detector produces unnecessary alarms before real drift occurs
Drift detection	Detection Count	Total number of ADWIN alarms during the simulation
Prediction accuracy	MAE	Straightforward indicator of prediction accuracy.
Prediction accuracy	RMSE	Identifies significant deviations; weights larger errors more heavily than MAE.
Prediction accuracy	MAPE	Enables comparison across different scales; proportional error measure.
Model diagnostics	Residual Mean	Shows systematic bias in prediction; values near zero indicate low average over or under prediction.
Model diagnostics	Residual Standard Deviation	Measures the spread of prediction errors; lower values indicate more stable prediction performance.
Model diagnostics	Residual Analysis	The model has captured the underlying data structure.
Bottleneck forecasting	F1 Score	Bottleneck detection performance measure.

Bottleneck forecasting	Precision	Proportion of predicted bottleneck alerts that are correct; low precision indicates excessive false alarms.
Bottleneck forecasting	Recall	Proportion of actual bottleneck events correctly identified; low recall indicates missed bottlenecks.
bottleneck forecasting	Balanced Accuracy	Accounts for class imbalance in bottleneck classification; more reliable than standard accuracy on imbalanced data.
bottleneck forecasting	Lead Time	Number of steps between the first model warning and the actual bottleneck event.

2.7 State estimation

State estimation in this study refers to identifying the current operating condition of the production process from observed time-based variables. In the forecasting pipelines, this operating condition is not represented through explicit machine-state labels such as Processing, Starved, Blocked or Down. Instead, it is represented indirectly through production variables that describe the recent state of the system. In the cycle time prediction pipeline, the current process state is represented by sequential observations of Processing Time, Energy Consumption, Machine Availability and stream Ligand Station ID. These variables provide the operational context used by the LSTM model to estimate the near-future cycle time state of the system. In addition, the ADWIN mechanism monitors the squared prediction error of the adaptive LSTM model to detect whether the estimated process behaviour has changed significantly over time. In the bottleneck forecasting pipeline, state estimation is represented through the combined use of Machine Availability, Processing Time, Energy Consumption, and Upstream_Lag factorised Machine ID. These variables describe the recent operating condition of the line and are used to estimate whether the system is moving toward a bottleneck state. Bottleneck labels are assigned when Processing Time exceeds the 85th percentile and

Machine Availability falls below the 15th percentile at the same time, or when Processing Time alone exceeds the 95th percentile. This provides an estimated binary operational state for bottleneck forecasting. To maintain valid state estimation under changing conditions, adaptive monitoring is applied in both pipelines. In the cycle time prediction pipeline, ADWIN is used to detect significant changes in the prediction-error pattern and trigger retraining of the adaptive LSTM model. In the bottleneck forecasting pipeline, recent model performance is monitored through the F1 score over a short history window. When this score falls below 0.65, retraining is triggered on recent data. In the bottleneck pipeline, recent F1 score is monitored over a short history window and retraining is triggered when it falls below 0.65. This rule is active only during the drift interval, with a minimum 40 step gap enforced between successive retraining events.

3 IMPLEMENTATION, INTEGRATION

In this study, integration refers to how different components of the forecasting system data processing, predictive models, drift detection mechanisms and evaluation logic are combined into a single operational framework. Implementation refers to how these components are practically developed, structured and executed within the two forecasting pipelines. It includes cycle time prediction and bottleneck forecasting. The implementation begins with a unified data pipeline. The system acquires production data from the manufacturing dataset and prepares it through cleaning, feature extraction, and sequence generation. The same dataset is used for both tasks, but the implementation differs in feature selection, target definition and normalisation. Sequential input windows of 10-time steps are constructed. It uses a sliding-window approach, allowing both pipelines to operate on time-ordered production data.

The integration of models is task specific. In the cycle time prediction pipeline, ARIMA and standalone LSTM are implemented as baseline models. The adaptive model consists of an adaptive LSTM combined with ADWIN. In the bottleneck forecasting pipeline, static LSTM models serve as baselines and adaptive LSTM models are implemented separately. It is deployed for each prediction horizon of one, three and five steps ahead. This structure allows direct comparison between traditional, non-adaptive and adaptive approaches under identical conditions. The key integration element is the Adaptive Intelligence Layer. It connects prediction outputs with update decisions. In the cycle time pipeline, ADWIN is integrated as a drift detection mechanism that monitors the squared prediction error stream. When a distributional change in the error stream is detected, the system triggers retraining of the adaptive LSTM. It uses a recent data window, with feature scaling recalculated for that subset. In the bottleneck pipeline, integration is achieved through a performance-based mechanism. The system monitors recent F1 score, and retraining is triggered when it falls below 0.65. During the drift interval, subject to a minimum gap of 40 steps. Retraining is performed on the most recent 70step data window without recalibrating the scaler.

The implementation also integrates decision logic into the prediction process. In bottleneck forecasting, predicted probabilities are converted into binary alerts. It uses phase dependent thresholds of 0.70 in the stable phase, 0.55 in the drift phase and 0.45 in the recovery phase. According to operating conditions, the system adjusts its alert sensitivity. In contrast, the cycle time pipeline produces continuous outputs and focuses on error monitoring. The drift triggered updates rather than threshold based decisions. Finally, the system integrates evaluation and reporting across all operating phases. Performance is assessed separately under stable, drift and recovery conditions with specific KPIs. For cycle time prediction, MAE, RMSE and MAPE are used together with detection delay and false positives. For bottleneck forecasting, evaluation includes F1 score, precision, recall, balanced accuracy and lead time. This phase-based evaluation framework supports consistent assessment of model behaviour. It directly contributes to answering the research questions by linking model performance.

3.1 Requirements definition

The functional requirements cover the main operations of the implemented forecasting system. The system acquires and preprocesses production data, organises it into 10step sequential input windows and applies the relevant input features for each task processing time, energy consumption, machine availability, upstream lag and station or machine identifiers. Both baseline and adaptive models are required for each task. For cycle time prediction, these are ARIMA and standalone LSTM as baselines and LSTM+ADWIN as the adaptive model. For bottleneck forecasting, these are a static LSTM as the baseline and an adaptive LSTM for each forecast horizon. Each pipeline includes a drift aware mechanism. In the cycle time pipeline, ADWIN monitors prediction error and triggers retraining when drift is detected. In the bottleneck pipeline, recent F1 score is monitored, and retraining is triggered when it falls below 0.65 during the drift interval. The system also generates prediction outputs, bottleneck warnings and KPI based evaluation results across stable, drift and recovery phases (see TABLE 2).

TABLE 2. Functional requirements

Requirement	Description
Production data acquisition	Acquire time series production data from the assembly line, including variables required for cycle time prediction and bottleneck forecasting, such as processing time and machine identifiers.
Data preprocessing	clean and preprocess the acquired data by handling missing values, selecting relevant variables and preparing the dataset for sequential forecasting tasks.
Sequential data handling	organise production observations into fixed-length input sequences for sort term forecasting. In the implemented models, sequence windows are used to predict the next time step.
Feature preparation	Transform raw production data into model input features suitable for forecasting.
Baseline model implementation	Implement baseline models for comparison. For cycle time prediction, these shall include ARIMA and standalone LSTM. For bottleneck forecasting, these shall include static LSTM and adaptive LSTM
Drift aware model integration	Integrate a concept drift detection method, such as ADWIN, with an adaptive LSTM-based forecasting pipeline.
Drift detection and adaptive update	Monitor incoming prediction errors for concept drift and trigger model updating or retraining when drift is detected.
Short term cycle time prediction	Generate short term forecasts of cycle time under dynamic operating conditions.
Short term bottleneck forecasting	Forecast short term bottleneck events in the assembly line as an Early warning function across multiple

	forecast horizons. In the implemented model, predictions are generated at one, three and five steps ahead
Bottleneck target definition	A bottleneck event is labelled when Processing_Time exceeds the 85th percentile and Machine_Availability falls below the 15th percentile simultaneously, or when Processing_Time alone exceeds the 95th percentile of the simulated process series.
Multi-phase model evaluation	Evaluate forecasting performance across different operating phases, including stable, drift and recovery conditions.
KPI reporting	Report relevant KPIs for model performance and drift behaviour, including MAE, RMSE, MAPE, detection delay, false positives and residual based evaluation where applicable.
Operational output generation	Provide operationally usable outputs, including predicted cycle times, predicted bottleneck events, drift alarms and model performance summaries.

The nonfunctional requirements define the expected quality of the system under operation. The system maintains acceptable prediction accuracy across stable, drift and recovery phases. It performs without performance collapse during phase transitions. Requirements cover accuracy, adaptability, timeliness, stability, reliability and maintainability. Limiting false alarms is also required, as excessive false detections reduce the operational usefulness of a drift aware warning system addressed in the cycle time pipeline through explicit false positive reporting and in the bottleneck pipeline through phase-dependent decision thresholds of 0.70, 0.55 and 0.45. These requirements are evaluated through MAE, RMSE and MAPE for cycle time prediction and F1 score, precision, recall, balanced accuracy and lead time for bottleneck forecasting, together with detection delay and false positives for drift detection (see TABLE 3).

TABLE 3. Nonfunctional requirements

Requirement	Description
Prediction accuracy	Achieve measurable predictive performance using relevant error metrics for cycle time prediction and F1 score, precision, recall, balanced accuracy and lead time for bottleneck forecasting
Robustness under non stationarity	Maintain acceptable performance under changing operating conditions, including drift, regime shifts and recovery periods.
Adaptability	Adapt to evolving data distributions through drift detection and selective model retraining.
Timeliness	Provide short term forecasts and bottleneck alerts early enough to support operational intervention.
Stability across operating phases	Maintain reasonable performance consistency across stable, drift and recovery phases.
Reliability	Operate reliably in repeated monitoring and prediction cycles over incoming production data.
Low false-alarm behaviour	Minimize false drift alarms and false bottleneck warnings in order to preserve operational usefulness.
Scalability to sequential data flow	Support repeated prediction and updating as new production observations become available over time.
Maintainability	Framework allows retraining and replacement of predictive models without redesigning the entire system.
Operational interpretability	The system outputs understandable through forecast values, bottleneck-event predictions, phase-based KPI reporting and drift indicators.

The domain specific requirements reflect the fact that the system is intended for an industrial assembly line setting rather than a general forecasting problem. The model handles compatible with manufacturing data and with the sequential nature of production flow. The requirements therefore emphasise the use of production variables that are meaningful in assembly line analysis, including processing time, machine availability, energy related information, upstream flow characteristics and station-level identifiers. The table also states that bottlenecks must be represented in a way that matches actual line behaviour. In the implemented bottleneck forecasting setup, a bottleneck event is defined through a threshold-based rule based on processing time and machine availability, together with an additional extreme processing-time condition, which allows the forecasting task to be formulated as a sort of term classification problem. The domain specific requirements also highlight the need for continuous monitoring, Early warning capability and practical suitability for deployment in environments where process behaviour may change over time (see TABLE 4).

TABLE 4. Domain specific requirements

Requirement	Description
Industrial assembly line applicability	Designed for an industrial assembly line context with dynamic and changing operating conditions.
Manufacturing data compatibility	Use production variables relevant to assembly line analysis, including processing time, energy consumption, upstream flow information, station or machine identifiers and related sequential observations.
Flow aware modelling	The forecasting approach incorporates assembly line flow information where available. In the implemented bottleneck model, this is represented through upstream lag and station-level identifiers.
Bottleneck representation	It is a sort of term production constraint using threshold-based rules on processing time and machine availability, plus an extreme processing-time condition, forming a binary classification target

Support for dynamic operating conditions	Remain applicable under changing production conditions, including drift, disturbance and recovery scenarios.
Comparative industrial relevance	Support comparison between traditional forecasting approaches and adaptive machine learning approaches in an assembly line setting.
Industrially relevant evaluation	Performance is assessed using MAE, RMSE and MAPE for cycle time prediction; F1 score, precision, recall, balanced accuracy and lead time for bottleneck forecasting; and detection delay and false positives for drift detection.
Early warning capability	Forecasting function shall support prediction before bottleneck conditions fully materialise in the production line.
Continuous monitoring in deployment	Support ongoing monitoring of production data and repeated reassessment of model validity during operation.
Deployment suitability	It designs for practical use in industrial assembly lines where process behaviour changes over time and predictive outputs must support operational decision-making.

3.2 Dataset definition

This thesis used the manufacturing production dataset (Ziya 2023). The dataset contains 1,000 observations and 13 variables, each record covering job identification, machine assignment, operation type, material usage, processing duration, energy consumption, machine availability, scheduled and actual execution times and job status classification. The complete list of variables includes Job ID, Machine ID, Operation Type, Material Used, Processing Time, Energy Consumption, Machine Availability, Scheduled Start and End, Actual Start and End, Job Status and Optimisation Category.

3.3 Data acquisition and processing pipeline

This section describes how the raw dataset was prepared and structured for use in both forecasting tasks. The pipeline includes data loading, data cleaning, feature extraction, sequence generation and the definition of the three evaluation phases. The same dataset is used for cycle time prediction and bottleneck forecasting, and it is different in feature selection, target construction and normalisation strategy between the two tasks. Of the 13 variables, the primary model inputs are Processing Time, Energy Consumption and Machine ID. Three additional variables are constructed during preprocessing for the bottleneck forecasting task: Station ID, Upstream Lag, and Bottleneck Event

The remaining variables Scheduled Start, Scheduled End, Actual Start, Actual End, Job Status and Optimisation Category provide contextual information about scheduling, execution timing and job condition. Missing values appear only in Actual Start and End, each with 129 missing entries. Since these variables are not used as model inputs or targets, no imputation is applied, and all 1,000 records are retained. Rows where the derived variable Upstream Lag contains missing values are excluded before model training, leaving 995 records. Processing Time is the cycle time prediction target, recording the duration of each manufacturing operation. It ranges from 20 to 120-time units with a mean of 71.38. A standard deviation of 28.48 and a median of 72. For the bottleneck forecasting, the target is a binary variable derived from the simulated Processing Time series. The compound threshold rule described in Section 3.3.3 is used. Machine ID identifies the five machines, M01 through M05. Operation Type covers five categories including Grinding, Additive, Lathe, Milling and Drilling. The scheduling and execution time variables supply sequencing information across the line. The Station ID and Upstream Lag support the representation of short-term production flow behaviour in the bottleneck forecasting.

3.3.1 Data loading

The dataset contains 1000 records and 13 variables representing production jobs processed across five machines from M01 to M05 and five operation types including Additive, Drilling, Grinding, Lathe and Milling. The file is loaded directly using a standard tabular read operation. No transformation is applied at the loading stage (see TABLE 5).

TABLE 5. Dataset summary

File	hybrid_manufacturing_categorical.csv
Total records	1,000
Total variables	13
Machines	M01, M02, M03, M04, M05
Operation types	Additive, Drilling, Grinding, Lathe, Milling
Variables	Job_ID, Machine_ID, Operation_Type, Material_Used, Processing_Time, Energy_Consumption, Machine_Availability, Scheduled_Start, Scheduled_End, Actual_Start, Actual_End, Job_Status, Optimisation_Category

3.3.2 Data cleaning

Among the 13 variables, 11 contain no missing data. In the dataset, missing values are only present in the Actual_Start and End columns, both of which have 129 empty entries. Since these specific variables are not required as inputs or targets for answering questions, imputation techniques and record removal were not applied. The core model inputs Processing_Time, Energy_Consumption, Machine_Availability and, Upstream_Lag it contains no missing values in the original dataset. Upstream_Lag is derived by shifting Processing_Time one position per machine group. It introduces five null entries at the start of each machine sequence. These five

rows are removed then leaving 995 records for model training and finally evaluation (see TABLE 6).

TABLE 6. Data cleaning

Actual_Start missing	129 values, variable not used in any model
Actual_End missing	129 values, variable not used in any model
Upstream_Lag missing	5 values, first record of each of 5 machine groups
Action taken	Remove 5 rows with missing Upstream_Lag
Records after cleaning	995

3.3.3 Feature extraction

Two additional variables are derived during preprocessing. Upstream_Lag is the previous Processing_Time these values are used for the same machine. It is capturing short term upstream production behaviour. Station_ID is a numeric encoding of Machine_ID, it is allowing machine identity for model input. For cycle time prediction, the five input features in short-term forecasting, the aim is to predict bottleneck conditions before they fully develop, rather than identifying them only after they have already occurred.

Upstream_Lag and Station_ID are used as input features, with the next-step Processing_Time serving as the target. For bottleneck forecasting, the four input features are factorised Machine_ID, Energy_Consumption, Upstream_Lag and Machine_Availability. Processing_Time is excluded as an input because it defines the bottleneck target. The bottleneck target is stored as drifted target and recalculated from the simulated process and availability series after simulation. A record is labelled as a bottleneck when Processing_Time exceeds the 85th percentile and Machine_Availability falls below the 15th percentile simultaneously, or when Processing_Time alone exceeds the 95th percentile of the simulated series. This target is used for prediction at $t+1$, $t+3$ and $t+5$. Processing_Time is not used directly as an input because it is used to define the bottleneck target. The bottleneck target is created

after simulation and stored as drifted target. A record is labelled as a bottleneck if Processing_Time is above the 85th percentile and Machine_Availability is below the 15th percentile, or if Processing_Time is above the 95th percentile of the simulated series. This target is used for bottleneck prediction at $t+1$, $t+3$ and $t+5$. In the cycle time prediction model, the feature matrix includes five variables feature matrix includes five variables Processing_Time, Energy_Consumption, Machine_Availability, Upstream_Lag and Station_ID. These are used as sequential inputs to predict the next Processing_Time value (see TABLE 7).

In the bottleneck forecasting model, the feature matrix includes four variables factorised Machine_ID, Energy_Consumption, Upstream_Lag and Machine_Availability. Processing_Time is not used directly as an input because it is used to define the bottleneck target. The bottleneck target is stored as drifted_target. A record is labelled as a bottleneck if Processing_Time is above the 85th percentile and Machine_Availability is below the 15th percentile, or if Processing_Time is above the 95th percentile of the simulated process series. This target is used for bottleneck prediction (see TABLE 7).

TABLE 7. Input feature creation

Item	Description
Derived variable 1	Upstream_Lag = previous Processing_Time for the same Machine_ID
Derived variable 2	Station_ID = integer encoding of Machine_ID
Cycle time prediction features (5)	Processing_Time, Energy_Consumption, Machine_Availability, Upstream_Lag, Station_ID
Derived variable 1	Upstream_Lag = previous Processing_Time for the same Machine_ID
Derived variable 2	Station_ID = integer encoding of Machine_ID
Cycle time prediction features (5)	Processing_Time, Energy_Consumption, Machine_Availability, Upstream_Lag, Station_ID

Derived variable 1	Upstream_Lag = previous Processing_Time for the same Machine_ID
Derived variable 2	Station_ID = integer encoding of Machine_ID

3.3.4 Sequence generation

The records are organised into sequential inputs using a sliding-window approach. In both implementations, each input window contains 10 consecutive time steps. In the cycle time prediction implementation, each input sequence contains 5 features: Processing_Time, Energy_Consumption, Machine_Availability, Upstream_Lag and Station_ID. The model uses these 10-step windows to predict the next step Processing_Time value ($t+1$). The input is kept in LSTM tensor form with shape (10, 5). Before sequence generation, the feature matrix and target series are scaled using Standard Scaler fitted on the first 400 records, which form the initial training phase. During adaptive retraining, the scaler is recalculated on the current refitting subset, and the adaptive LSTM is retrained using sequences generated from that rescaled subset. The bottleneck forecasting pipeline uses four input features factorised Machine_ID, Energy_Consumption, Upstream_Lag and Machine_Availability organised into tensors of shape (10, 4). A single StandardScaler is fitted on records 0 – 399 and reused throughout evaluation without refitting. Separate training sequences are generated for each forecast horizon $t+1$, $t+3$ and $t+5$ and both the static and adaptive LSTM models use the same sequence structure (see TABLE 8).

TABLE 8. Sequence generation

Item	Description
Window length	10-time steps
Cycle time prediction forecast horizon	$t+1$
Bottleneck forecasting horizons	$t+1$, $t+3$ and $t+5$

Cycle time prediction input shape	(10, 5) tensor for LSTM
Bottleneck forecasting input shape	(10, 4) tensor for LSTM
Cycle time prediction normalisation	StandardScaler fitted on records 0–399 for initial training; during adaptive retraining, scaling is recalculated on the current refitting subset
Bottleneck forecasting normalisation	StandardScaler fitted once on records 0–399 and reused throughout evaluation
Cycle time prediction sequence target	Next step Processing_Time
Bottleneck forecasting sequence target	future drifted_target at horizon t+1, t+3, or t+5

3.3.5 Definition of stable, drift and recovery phases

After cleaning, the dataset contains 995 records divided into sequential phases for training and evaluation. Records 0–399 form the training phase, used only for initial model fitting covering ARIMA, standalone LSTM and the initial adaptive LSTM in the cycle time pipeline and the initial static and adaptive LSTM models for each forecast horizon in the bottleneck pipeline. No evaluation results are reported from this phase. In the bottleneck forecasting implementation, it is used to fit the initial static and adaptive LSTM models for each forecast horizon. No evaluation results are reported from this phase. Records 400 to 599 form the stable phase. In this phase, the original process values are kept unchanged. This phase represents normal operating conditions and provides the baseline for later comparison. Records from 600 to 799 are assigned to the drift phase.

In the cycle time prediction implementation, Processing_Time is multiplied by 1.6 and produce a sustained increase in cycle time. In the bottleneck forecasting implementation, Processing_Time is multiplied by 1.8 and Machine_Availability is multiplied by 0.4. This phase represents the main simulated regime change and is used to evaluate adaptive model behaviour under drift. Records 800 onward form the recovery phase. In the cycle time prediction implementation, Processing_Time is multiplied by 1.15 relative to the original

values. In the bottleneck forecasting implementation, `Processing_Time` is multiplied by 1.3 relative to the original values. This phase represents a partially recovered operating condition after the main drift period. The original dataset does not contain explicit natural concept drift. Therefore, drift is introduced experimentally through controlled changes in `Processing_Time` and, in the bottleneck forecasting code, also in `Machine_Availability`. This creates the non-stationary conditions required for evaluating adaptive forecasting behaviour (see TABLE 9). In the bottleneck forecasting implementation, the binary target is recalculated after simulation. The target is stored as `drifted_target` and is based on the simulated process and availability values. As a result, the class distribution changes across phases and becomes part of the dynamic operating condition being evaluated. Because the drift is simulated rather than observed directly in the raw production data, all reported results describe model behaviour under controlled experimental conditions (see TABLE 9).

TABLE 9. Definition of stable, drift and recovery phases

Item	Description
Records 0–399	Training phase, initial model fitting only; no evaluation
Records 400–599	Stable phase original values retained
Records 600–799 (cycle time prediction)	Drift phase $\text{Processing_Time} \times 1.6$
Records 800–994 (cycle time prediction)	Recovery phase $\text{Processing_Time} \times 1.15$ relative to original values
Records 600–799 (bottleneck forecasting)	Drift phase $\text{Processing_Time} \times 1.8$ and $\text{Machine_Availability} \times 0.4$
Records 800–899 (bottleneck forecasting evaluation)	Recovery phase $\text{Processing_Time} \times 1.3$ relative to original values
Drift source	Experimentally introduced simulation, not naturally occurring drift in the raw dataset

Bottleneck target after simulation	Recalculated as drifted_target from the simulated process and availability series
------------------------------------	---

3.4 Baseline, model implementation

Two different baseline setups are used in this study, one for cycle time prediction and one for bottleneck forecasting. In the cycle time prediction implementation, the baseline models are ARIMA and standalone LSTM. ARIMA is used as a traditional time series forecasting approach. At each evaluation step, it is fitted on the most recent 50 observations of Processing_Time and then used to forecast the next value. The standalone LSTM model is trained on the initial training phase using sequential windows of length 10 and input shape (10, 5). Its role is to provide a machine-learning baseline without drift adaptation. The adaptive version starts from the same initial weights as the standalone LSTM and is later updated when the ADWIN detector signals drift. In the bottleneck forecasting implementation, the baseline model is the static LSTM. This model uses sequential windows of length 10 and input shape (10, 4) to predict future bottleneck events at $t+1$, $t+3$ and $t+5$. Separate static models are trained for each forecast horizon using the initial training phase. The adaptive model is initialised with the same weights as the corresponding static model and is then selectively retrained during evaluation when recent predictive performance drops below the defined condition. In this implementation, the static LSTM serves as the non-adaptive baseline for comparison with the adaptive bottleneck predictor. Both baseline setups are evaluated under stable, drift and recovery conditions. This allows direct comparison between traditional forecasting, non-adaptive sequence learning and adaptive sequence learning under changing operating regimes.

3.5 Adaptive intelligence layer

The Adaptive Intelligence Layer is the part of the system that allows the forecasting models to respond to changing operating conditions instead of remaining fixed after initial training. In

this study, it is implemented differently in the two pipelines, but both pipelines share the same goal monitor recent behaviour, detect change and update when needed. In the cycle time prediction pipeline, the Adaptive Intelligence Layer is built around a custom ADWIN drift detector and an adaptive LSTM model. After the initial training phase, the adaptive LSTM produces rolling next step forecasts of `Processing_Time`. At each step, the squared prediction error is sent to the ADWIN detector. ADWIN monitors the error stream and checks whether the mean error in the current window has changed enough to indicate drift. When drift is detected during or after the drift phase, the detection step is logged, and the adaptive LSTM is retrained on a recent subset of the data.

This retraining uses newly scaled data from the selected context window, and the feature and target scalers are recalculated for that subset before refitting. In this way, the Adaptive Intelligence Layer combines drift detection, local data selection, rescaling and model updating in one continuous loop. In the bottleneck forecasting pipeline, the Adaptive Intelligence Layer is implemented as a performance-triggered adaptive retraining mechanism. Separate adaptive LSTM models are maintained for the three bottleneck horizons: $t+1$, $t+3$ and $t+5$. During evaluation, each adaptive model maintains a short history of recent actual labels and predicted probabilities. Within the drift interval, the F1 score is calculated on this history using a fixed monitoring threshold of 0.5 and retraining is triggered for that horizon when the score falls below 0.65 and at least 40 steps have passed since the last update. The adaptive model is then retrained on the most recent 70step window using the same fixed scaler fitted on the initial training phase. This means that, in the bottleneck pipeline, the Adaptive Intelligence Layer is responsible for performance monitoring, retraining decision logic and horizon-specific model updating. The Adaptive Intelligence Layer also includes the decision rules used at prediction time. In the bottleneck pipeline, predicted probabilities are converted into bottleneck alerts using phase-aware thresholds 0.70 in the stable phase, 0.55 in the drift phase and 0.45 in the recovery phase. In the cycle time pipeline, the adaptive output remains a continuous prediction and the layer focuses on error monitoring and residual based drift handling. Across both pipelines, the Adaptive Intelligence Layer is the mechanism that keeps the forecasting framework responsive to non-stationary production conditions driven by ADWIN based drift detection in the cycle time model and by performance-triggered retraining in the bottleneck

model. In the bottleneck model, it is driven by recent-performance monitoring and selective horizon-specific retraining. The layer connects prediction to model update, providing the mechanism for adaptation across all phases. The simulation environment is designed to evaluate model behaviour under controlled changes in assembly line operating conditions. Since the original dataset does not contain explicit natural concept drift, both implementations introduce drift artificially to test forecasting accuracy, adaptation and robustness under non-stationary conditions. The dataset used in both implementations are the same. After preprocessing and removal of rows with missing derived lag values, the simulations operate on the cleaned sequential production records.

In both codes, the records are kept in time order so that the production sequence is preserved during training and evaluation. In the cycle time prediction simulation, the process variable `Processing_Time` is modified directly to create regime changes. The data is divided into a training phase, a stable phase, a drift phase and recovery phase. During the drift phase, `Processing_Time` is multiplied by 1.6 and during the recovery phase it is multiplied by 1.15 relative to the original values. The adaptive model then generates rolling next step forecasts while the ADWIN mechanism monitors squared prediction error and triggers retraining when drift is detected. In the bottleneck forecasting simulation, both `Processing_Time` and `Machine_Availability` is modified to create operational disruption. During the drift phase, `Processing_Time` is multiplied by 1.8 and `Machine_Availability` is multiplied by 0.4. During the recovery phase, `Processing_Time` is multiplied by 1.3 relative to the original values. In this implementation, `Machine_Availability` is not only modified during simulation but is also included as an input feature together with factorised `Machine_ID`, `Energy_Consumption` and `Upstream_Lag`. The bottleneck target is then recalculated from the simulated series and stored as drifted target. Its definition is compound. A record is labelled as a bottleneck if either `Processing_Time` is above the 85th percentile of the cleaned `Processing_Time` distribution and `Machine_Availability` is below the 15th percentile of the cleaned `Machine_Availability` distribution, or if `Processing_Time` is above the 95th percentile of the simulated process series. The simulation runs in a rolling manner at each step, the models receive the most recent 10 step input window and produce a forecast. Static models remain fixed after initial training. Adaptive models update during evaluation: in the cycle time pipeline, ADWIN triggers retraining based

on squared prediction error; in the bottleneck pipeline, retraining is triggered by recent F1 score within the drift interval. The cycle time simulation evaluates continuous value prediction under drift, while the bottleneck simulation evaluates short term bottleneck forecasting under changing production conditions (see FIGURE 2). One methodological point requires clarification. Earlier project material defined the bottleneck target using a single Processing_Time threshold at the 90th percentile. The implemented code uses the compound rule Processing_Time above the 85th percentile and Machine_Availability below the 15th percentile, or Processing_Time above the 95th percentile of the simulated series. The implemented rule applies throughout this thesis (see FIGURE 2).

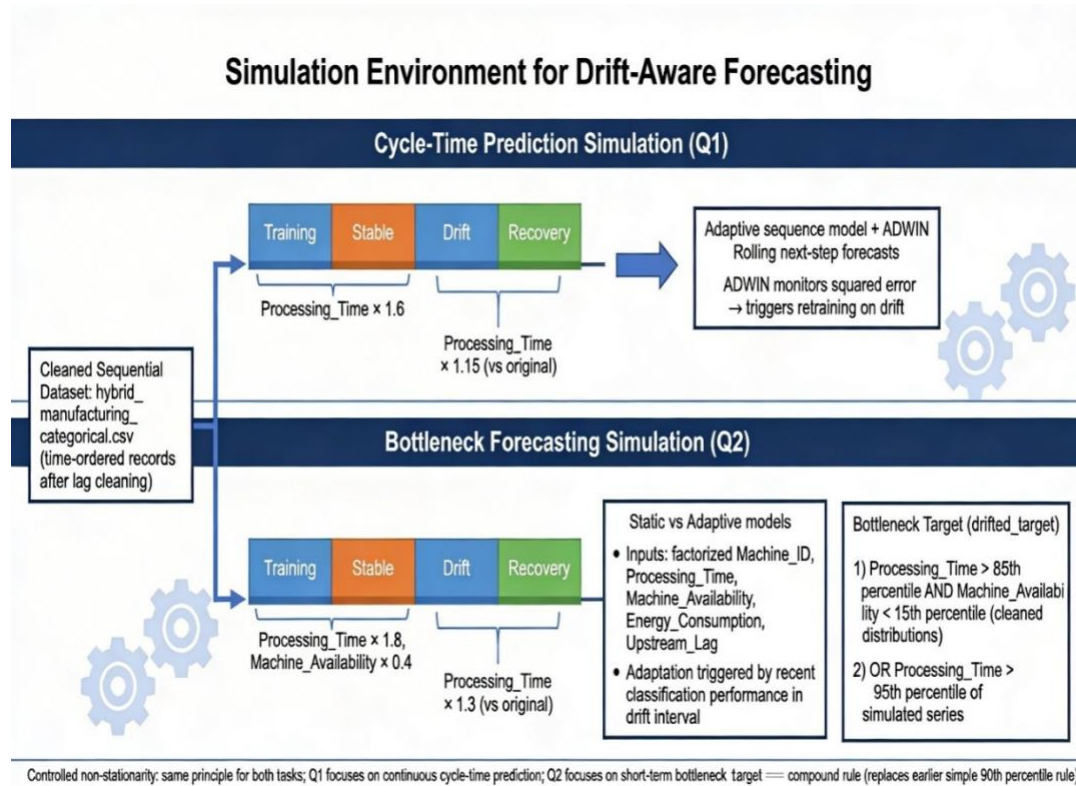


FIGURE 2. Simulation environment

3.6 Model evaluation and KPI reporting

Model evaluation is carried out separately for the two implemented tasks: cycle time prediction and bottleneck forecasting. Because the two tasks use different target types, they also use

different KPI sets. In the cycle time prediction implementation, the target is the continuous variable `Processing_Time`. The models are evaluated in three operating phases: stable, drift and recovery. For each phase, the predictions of ARIMA, standalone LSTM and LSTM+ADWIN are compared against the actual process values. The reported KPIs including MAE, which measures the average absolute difference between predicted and actual cycle time. RMSE, which gives more weight to larger prediction errors. MAPE, which expresses prediction error relative to the actual value. These metrics are calculated phase by phase using the rolling forecast outputs stored in `eval_df`. In the same implementation, drift-related KPIs are also reported. Detection delay is calculated as the number of steps between the start of the simulated drift phase and the first ADWIN detection within that phase. False positives are counted as ADWIN detections that occur before the drift phase begins. In addition, residual-based diagnostics are used for the adaptive LSTM. Residuals are examined through a distribution plot and a phase-based boxplot. It tracks how prediction error shifts across stable, drift and recovery conditions. In the bottleneck forecasting pipeline, the target is the binary variable drifted target, evaluated separately for each forecast horizon $t+1$, $t+3$ and $t+5$. For each horizon and each phase, the predictions of the static LSTM and adaptive LSTM are evaluated using: F1 score, Precision, Recall, Balanced Accuracy. The predicted probabilities are converted into binary bottleneck alerts using a threshold. In the static model, the threshold is fixed at 0.5. In the adaptive model, the threshold is phase-dependent: 0.7 in the stable phase, 0.55 in the drift phase, 0.45 in the recovery phase. These classification KPIs are calculated from the rolling prediction outputs stored in `eval_df` and summarized in summary rows. In this specific bottleneck implementation, MAE is not included in the KPI summary block.

The bottleneck implementation also reports lead time. Lead time is calculated for each forecast horizon by scanning the full evaluation period stored in `eval_df`. When a bottleneck event starts, identified by a change from 0 to 1 in the actual label sequence, the code checks the previous 10 steps for the first valid adaptive alert. If such an alert exists, lead time is measured as the number of steps between that alert and the event. The reported value is the average lead time across all detected events in the full evaluation period, not only within the drift phase. Because the adaptive model uses different decision thresholds in the stable, drift and recovery phases, the reported lead-time averages may include events generated under different phase-

specific threshold conditions. Overall, KPI reporting in the code is task specific. The cycle time implementation reports regression and drift-detection metrics, while the bottleneck implementation reports classification and lead-time metrics. This separation matches the two forecasting objectives implemented in the system.

3.7 Practical challenges and benefits

The implementation of drift aware forecasting models in an assembly line environment involves both practical benefits and operational challenges. The system developed in this study combines sequential data processing, adaptive LSTM models, drift detection through ADWIN and performance-triggered retraining into two operational forecasting pipelines one for cycle time prediction and one for bottleneck forecasting. While this design allows the system to respond to changing production conditions. It remains fixed after initial training; it also introduces practical requirements and constraints. It is considered during system evaluation and result interpretation.

These challenges are due to the gap between simulation and real industrial deployment. The drift conditions are experimentally imposed naturally observed and the reported results reflect behaviour under these specific conditions. The bottleneck pipeline relies on phase dependent decision thresholds and a fixed feature scaler. Both of which assume that the operating phase is known in advance and that the input data distribution remains stable across operating conditions. They may not hold in a live environment. Class imbalance introduced by the simulated process changes also affects the choice of evaluation metrics. In addition, the retraining mechanism introduces computational cost and timing constraints that aligned with the actual rate of process change. The benefits reflect the practical value of adaptive forecasting over non adaptive alternatives. The adaptive models respond to detected process changes by retraining on recent data. It maintains useful performance after operating conditions have shifted. The bottleneck pipeline provides warnings several steps before an event occur and gives operators a defined window for intervention. Phase dependent thresholds provide structured control over alert sensitivity and phase-based reporting. It allows performance to be

assessed independently under stable and recovery conditions. This section identifies each of these challenges in detail with direct reference to the implementation choices made in the two forecasting pipelines.

3.7.1 Challenges

Drift simulation versus real drift: the drift conditions tested in this study were implemented by scaling processing time by factors of 1.6 and 1.15 in the cycle time pipeline. In the bottleneck pipeline, `Processing_Time` was multiplied by 1.8 and `Machine_Availability` was multiplied by 0.4. These values were experimentally imposed, not naturally occurring. Real production environments produce gradual, mixed or irregular drift patterns. They may not trigger ADWIN or the F1 score rule as cleanly as the controlled simulation does. The detection delay and false positive counts reported here reflect behaviour on this dataset under these specific multipliers. Class imbalance in bottleneck forecasting: Under the implemented threshold rule, bottleneck events are identified from the simulated process and availability series using a compound criterion based on elevated `Processing_Time` and reduced `Machine_Availability`.

This creates class imbalance and changing class distribution across operating phases. Standard accuracy is therefore unreliable as a performance measure, which is why balanced accuracy and F1 score are used throughout the bottleneck evaluation. Phase-dependent threshold management: The bottleneck pipeline uses three different decision thresholds—0.70 in the stable phase, 0.55 in the drift phase, and 0.45 in the recovery phase to convert predicted probabilities into binary alerts in a live deployment, the current operating phase cannot be labelled in advance. A mechanism for detecting the current phase automatically, or for selecting the appropriate threshold without manual intervention, would be required. Scaler validity under drift: In the cycle time pipeline, the `StandardScaler` is recalibrated using the current retraining window whenever ADWIN detects drift. In the bottleneck pipeline, the scaler is fitted once on the first 400 records and reused throughout evaluation. If the statistical distribution of input features shifts substantially during the drift phase, the fixed scaler in the bottleneck pipeline may no longer represent the current data distribution accurately.

Retraining cost and timing: Each retraining event requires the training of a new LSTM model using a recent data window. Retraining in the bottleneck pipeline applies to three separate horizon specific models. In time sensitive production environments, the computational cost of retraining and the 40-step minimum gap between events align with the actual rate of process change. It avoids extend periods of degraded performance.

3.7.2 Benefits

Earlier bottleneck warnings: The adaptive LSTM generates bottleneck predictions for forecast horizons of $t+1$, $t+3$, and $t+5$. It led time measured as the number of steps between the first valid alert and the actual bottleneck. It provides operators with a predefined intervention window before throughput disruption occurs. The model demonstrated maintained performance under changing operating conditions. ARIMA and the static LSTM remain fixed after initial training. They accumulate prediction error without correction as the drift phase in `Processing_Time` and `Machine_Availability`. The adaptive LSTM in each pipeline responds by retraining on recent data. It updates models' performance under changing conditions. Threshold control through phase-aware decision rules. Phase-aware decision thresholds 0.70 in the stable phase, 0.55 in the drift phase and 0.45 in the recovery phase. It adjusts the sensitivity of bottleneck alerts across operating conditions rather than applying a fixed conversion rule. The proposed framework supports structured KPI reporting across different operational phases. The framework reports phase-specific metrics across all three evaluation phases including MAE, RMSE and MAPE for cycle time prediction. Performance was evaluated using F1-score, precision, recall, balanced accuracy, and lead time for bottleneck forecasting, together with detection delay and false-positive rates for drift detection. Reporting by phase rather than as a single aggregate score allows performance to be assessed independently under three conditions. It is informed decisions about model deployment and retraining policy.

4 EVALUATION, RESULTS AND REFINEMENT

This chapter presents the evaluation results of the two implemented forecasting tasks sort term cycle time prediction and sort term bottleneck forecasting. It examines model behaviour across stable, drift and recovery operating conditions. The evaluation is based on the experimental design described in Chapter 3, in which drift conditions were introduced through controlled simulation applied to a dataset. The chapter is structured as follows. Section 4.1 describes the experimental design, including the evaluation phases, scenario definition and data setup used in both pipelines. Section 4.2 presents the results for cycle time prediction, comparing the performance of ARIMA, standalone LSTM and drift aware LSTM with ADWIN across all three phases. Section 4.3 presents the results for bottleneck forecasting, including performance across forecast horizons, comparative analysis of the static and adaptive LSTM models and lead-time analysis. Section 4.4 examines drift aware behaviour and key performance indicator analysis, covering detection delay, false positives and residual diagnostics for the adaptive cycle time model. Section 4.5 addresses model refinement of adaptive updating logic, threshold refinement in bottleneck forecasting, accuracy versus stability and responsiveness versus false alarms. Section 4.6 revisits practical challenges and benefits considering the evaluation results and examines deployment-oriented considerations. Section 4.7 discusses the limitations of the study and the extent to which the findings may be generalised to other assembly line environments. Throughout the chapter, results are reported separately for each task and each operating phase. Phase-specific key performance indicators are used to assess prediction accuracy, detection reliability, alert quality and early warning capability. The purpose of this structure is to allow the findings to be interpreted in relation to each of the three research questions that guide the study.

4.1 Experimental design

The experimental design of this study is based on two forecasting tasks carried out on the same manufacturing dataset under controlled changes in operating conditions including short term cycle time prediction and sort term bottleneck forecasting both tasks use sequential production data and are evaluated under three operating phases: stable, drift and recovery.

4.1.1 Evaluation phases

As presented in Table 10, the dataset is divided into an initial training phase, a stable evaluation phase, a drift phase and recovery phase. (See TABEL 10.)

TABLE 10. Dataset dividing

Record range	Phase definition
0 – 399	Initial training
400 – 599	Stable evaluation phase
600 – 799	Drift evaluation phase
800 onward	Recovery phase in the cycle time prediction task
800 – 899	Recovery evaluation phase the bottleneck forecasting task

No evaluation results are reported from the training phase. Model evaluation begins only after the initial fitting stage has been completed.

4.1.2 Scenario definition

The evaluation is based on controlled simulation of changing production conditions. In the cycle time prediction task, the target variable is `Processing_Time`. During the drift phase, `Processing_Time` is multiplied by 1.6. During the recovery phase, it is multiplied by 1.15 relative to the original values. This creates a sustained change in cycle time and then a partial return toward the original operating level. In the bottleneck forecasting task, the target is a binary variable called `drifted_target`. During the drift phase, `Processing_Time` is multiplied by 1.8 and `Machine_Availability` is multiplied by 0.4. During the recovery phase, `Processing_Time` is multiplied by 1.3 relative to the original values. The bottleneck target is then recalculated from the simulated process and availability series. A record is labelled as a bottleneck if `Processing_Time` is above the 85th percentile of the cleaned `Processing_Time` distribution and `Machine_Availability` is below the 15th percentile of the cleaned `Machine_Availability` distribution, or `Processing_Time` is above the 95th percentile of the simulated process series. This design allows the models to be tested under normal operation, sustained disturbance and partial post drift recovery.

4.1.3 Data and sequence setup

In both tasks, the data is organised using 10step sliding windows so that the models use recent process history to generate short term forecasts. In the cycle time prediction task, the input features are `Processing_Time`, `Energy_Consumption`, `Machine_Availability`, `Upstream_Lag`, and `Station_ID`. The target is the next step value of `Processing_Time`. In this pipeline, `StandardScaler` is fitted on the initial training phase and used to scale both the feature matrix and the target series. When adaptive retraining is triggered, the scaler is recalculated on the selected retraining subset before the adaptive LSTM is refitted. In the bottleneck forecasting task, the input features are `Machine_ID`, `Energy_Consumption`, `Upstream_Lag` and

Machine_Availability. The model predicts future bottleneck events at three forecast horizons $t+1$, $t+3$ and $t+5$. In this pipeline, a single StandardScaler is fitted once on records 0–399 and reused throughout evaluation without refitting.

4.2 Results for cycle time prediction

The results for cycle time prediction evaluate the performance of three models ARIMA, standalone LSTM and LSTM with ADWIN across stable, drift and recovery phases using MAE, RMSE and MAPE. In the stable phase, where operating conditions remained unchanged, the standalone LSTM and LSTM with ADWIN produced identical results and both performed slightly better than ARIMA across all three error measures. This shows that under stable conditions, sequence-based learning provided marginally better accuracy than the statistical model. The adaptive mechanism did not provide any additional benefit in this phase, since no change in the data distribution occurred and both LSTM models produced the same values.

In the drift phase, where processing time was increased to simulate a sustained change in operating conditions, the results differed across the three error measures. ARIMA achieved the lowest MAE and RMSE. The standalone LSTM achieved the lowest mean absolute percentage error. LSTM with ADWIN did not produce the lowest value on any of the three error measures. These results indicate that model performance under drift depends on the selected error metric. It shows that the adaptive mechanism did not provide a clear accuracy advantage during the drift period itself. During the recovery phase, the process transitioned to a partially stabilised condition after the disturbance. LSTM with ADWIN achieved the strongest overall performance. It produced the lowest root mean squared error and mean absolute percentage error. The standalone LSTM achieved a slightly lower mean absolute error. ARIMA produced the highest errors across all three measures. These results show that the adaptive updating mechanism became most effective after the process had changed, allowing the model to better adjust to the new operating level. Across all three phases, no single model was best on every metric in every condition. The results demonstrate that model performance is dependent on the

operating phase and on the error, measure being considered. The adaptive LSTM provided the strongest overall performance in the recovery phase. The drift phase showed mixed results and the stable phase showed no difference between the standalone and adaptive LSTM models.

4.2.1 Performance in the stable phase

In the stable phase, the three models were evaluated under unchanged operating conditions using MAE, RMSE and MAPE. The results show that ARIMA achieved the best performance in this phase. Its error values were MAE = 0.0056, RMSE = 0.0070 and MAPE = 0.0033. The standalone LSTM model produced slightly higher errors MAE = 0.0063, RMSE = 0.0078, MAPE = 0.0038. LSTM with ADWIN model produced the same values as the standalone LSTM model in the stable phase MAE = 0.0063, RMSE = 0.0078 and MAPE = 0.0038. These results indicate that, under stable operating conditions, ARIMA provided the most accurate sort of term cycle time predictions among the three models. The standalone LSTM and the adaptive LSTM performed slightly worse, and the adaptive mechanism did not provide an additional advantage in this phase. This is consistent with the stable setting, where no major process change is present and therefore no adaptive update is required.

4.2.2 Performance in the drift phase

In the drift phase, the models were evaluated after the simulated increase in Processing_Time, where the production process moved into a disturbed operating condition. The results show a clear difference between the three forecasting approaches. The ARIMA model produced the highest prediction errors in this phase MAE = 0.0272, RMSE = 0.0336 and MAPE = 0.0163. The standalone LSTM model performed substantially better than ARIMA MAE = 0.0101, RMSE = 0.0125 and MAPE = 0.0053. The LSTM with ADWIN model achieved the lowest error values of all three models MAE = 0.0094, RMSE = 0.0116, MAPE = 0.0048. These results show that, under drift conditions, both LSTM models performed better than the

traditional ARIMA model. The lower error values of the standalone LSTM indicate that sequence-based learning was more effective than the statistical baseline when the process behaviour changed. The adaptive LSTM achieved a further reduction in error, which indicates that the integration of ADWIN improved sort term cycle time prediction during the drift phase. The difference between the two LSTM models is smaller than the difference between either LSTM model or ARIMA, but it is still consistent across all three error measures. This means that the adaptive mechanism contributed to better accuracy once the process entered the disturbed regime. In this phase, the adaptive model provided forecasting performance.

4.2.3 Performance in the recovery phase

During the recovery phase, model evaluation following the primary drift period showed that Processing_Time remained higher than its original baseline, though it did not return to the elevated levels seen during the drift phase. During the recovery phase, the forecasting models were evaluated to determine their performance after the process transitioned into a partially recovered state. The ARIMA model again produced the highest prediction errors MAE = 0.0249, RMSE = 0.0308, MAPE = 0.0163. The standalone LSTM model performed better than ARIMA MAE = 0.0116, RMSE = 0.0143, MAPE = 0.0076. LSTM with ADWIN model achieved the lowest error values in this phase MAE = 0.0089, RMSE = 0.0110, MAPE = 0.0058. These results show that both LSTM models continued to outperform the traditional ARIMA model after the drift period. The adaptive LSTM again achieved the best performance across all three error measures. This indicates that the adaptive updating mechanism remained beneficial. The process entered the recovery phase. The lower error values suggest that the adaptive model was better able to adjust to the new operating level after the disturbance than the non-adaptive alternatives. The difference between the adaptive and standalone LSTM models is also larger in the recovery phase than in the stable phase. The adaptive updating became more relevant once the process had changed. In this phase, the adaptive model provided the most accurate sort of term cycle time predictions.

4.2.4 Comparative results for ARIMA, standalone LSTM and adaptive LSTM

The comparative results in table 11 show how the three models performed across the stable, drift and recovery phases using Mean Absolute Error, Root Mean Squared Error and Mean Absolute Percentage Error. In the stable phase, the standalone LSTM model and LSTM with ADWIN model produced identical results, The ARIMA model produced slightly higher errors, this means that, under unchanged operating conditions, both LSTM models performed slightly better than ARIMA. The adaptive mechanism did not change the result in this phase, because the standalone and adaptive LSTM models had the same values. In the drift phase, the ARIMA model produced the lowest errors. For MAE and RMSE, ARIMA performed best. For MAPE, the standalone LSTM model performed best. The adaptive LSTM did not produce the lowest value on any of the three error measures in this phase. In the recovery phase, LSTM with ADWIN model produced the best results across all three metrics The ARIMA model produced the highest errors: In this phase, the adaptive model achieved the lowest MAPE and RMSE, while the standalone LSTM had a slightly lower MAE than the adaptive model. Overall, the adaptive model showed the strongest recovery-phase performance because it achieved the best result on two of the three reported error measures (TABLE 11).

TABLE 11. ARIMA, LSTM, LSTM+ADWIN compressive

Model	MAE	RMSE	MAPE
LSTM with Adaptive Windowing	26.8743	32.5352	0.4496
Standalone LSTM	26.8187	32.0104	0.4755
ARIMA	27.6465	33.2925	0.4791

Across the three phases, the comparative results show that model performance changes with operating conditions. In the stable phase, both LSTM models performed slightly better than ARIMA and the adaptive mechanism made no difference. In the drift phase, the results were mixed. ARIMA achieved the lowest MAE and RMSE, while standalone LSTM achieved the lowest MAPE. In the recovery phase, the adaptive LSTM produced the strongest overall

results, with the best RMSE and MAPE values. These results indicate that no single model was best on every metric in every phase. The comparison shows that model effectiveness depends on the operating condition.

4.3 Results for bottleneck forecasting

This section presents the results of short-term bottleneck forecasting under dynamic operating conditions. The evaluation compares the performance of the static and adaptive LSTM models across three forecast horizons of one, three and five steps ahead. It uses precision, F1 score, balanced accuracy and lead time. The results show that both models can forecast short term bottlenecks under changing conditions. They show different detection behaviours. The adaptive model achieves higher precision and provides early warnings before bottleneck events occur. The static model achieves higher overall detection performance in terms of F1 score. The model advantage depends on the evaluation metric and balance between alert reliability and comprehensive detection coverage. The analysis is structured across four subsections. Bottleneck class behaviour across phases. Performance at forecast horizons, Comparative results for the static and adaptive models and lead time analysis.

4.3.1 Bottleneck class behaviour across phases

In the Stable Phase, all three models produced similar results, which shows that cycle time prediction was performed under relatively unchanged operating conditions. The reported values were ARIMA: MAE = 24.8614, RMSE = 29.0270, MAPE = 0.4689, Standalone LSTM: MAE = 24.0926, RMSE = 28.0673, MAPE = 0.4466 and LSTM+ADWIN: MAE = 24.0926, RMSE = 28.0673, MAPE = 0.4466. These results show that the Standalone LSTM and LSTM+ADWIN models performed slightly better than ARIMA on all three error measures. Both LSTM-based models had lower MAE, lower RMSE and lower MAPE than ARIMA, which means they predicted the next step cycle time more accurately in this phase. The Standalone LSTM and LSTM+ADWIN models produced the same values. This means that, in

the Stable Phase, the adaptive mechanism did not change the forecasting result. The adaptive model behaved the same as the standalone LSTM under normal operating conditions. Overall, the outcome of the Stable Phase is that the two LSTM-based models achieved the best prediction accuracy, while ARIMA produced slightly higher error values.

4.3.2 Performance at forecast horizons $t+1$, $t+3$ and $t+5$

This analysis examines how bottleneck forecasting performance changes across different prediction horizons namely $t+1$, $t+3$ and $t+5$. Both the predictive capability and the limitations of time-series machine learning models are highlighted under dynamic operating conditions. During the drift phase, the Adaptive model consistently achieves higher precision than the Static model across all three forecast horizons. It means that bottleneck warnings are more accurate in terms of the proportion of alerts. At $t+1$ the Static model has a precision of about 0.69. The adaptive model reaches about 0.79 at $t+3$. In the static model is about 0.70 and the adaptive model is about 0.80 at $t+5$. The static model returns to about 0.69, while the adaptive model remains at about 0.80. These results show a stable advantage of roughly 0.10 to 0.11 in favour of the adaptive model across horizons. During the drift phase, bottleneck warnings from the Adaptive model are more correct than those from the static model. The precision reflects alert reliability and a model with higher precision. It generates a lower proportion of false bottleneck alarms among its issued alerts. In an assembly line setting. It repeated false warnings may reduce operator trust and result in unnecessary interventions. Adaptive model maintains nearly the same precision level from $t+1$ to $t+5$. It shows that its alert reliability remains stable across the sort term forecasting range used in the study. At the same time, the higher precision of the adaptive model must be interpreted in conjunction with the model's decision rule during the drift phase. In this implementation, the adaptive model uses a stricter threshold during drift than the static model. It is more conservative in deciding when to raise a bottleneck alert. This stricter threshold is consistent with the observed result, fewer alerts are likely to be raised, but a larger proportion of them are correct. Therefore, the precision results indicate that under dynamic operating conditions, the Adaptive model provides more reliable bottleneck warnings than the Static model (see FIGURE 3).

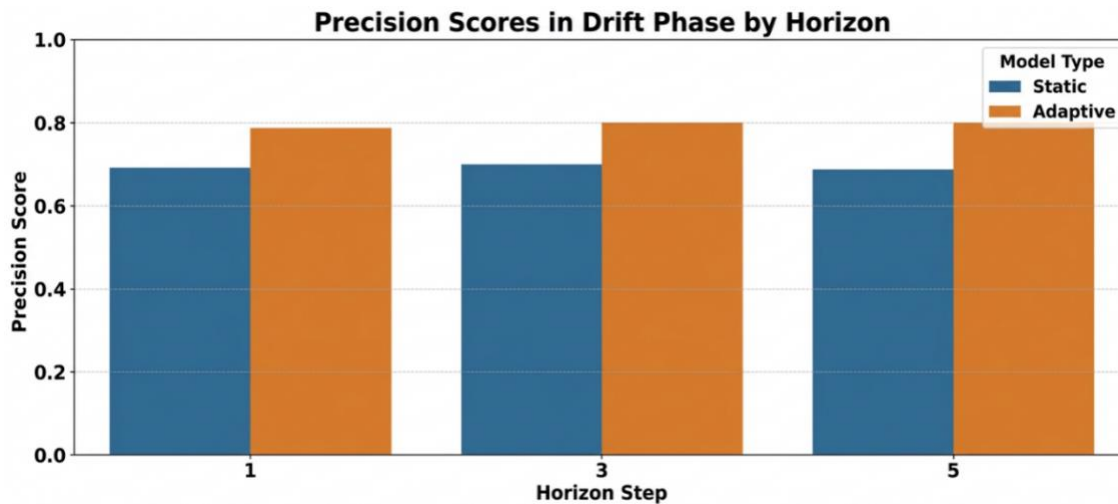


FIGURE 3. Precision score in drift phase

In the drift phase, the Static model achieves higher F1 scores than the Adaptive model across all three forecast horizons. The Static model has F1 scores of about 0.57 at $t+1$, 0.33 at $t+3$ and 0.54 at $t+5$. The Adaptive model has F1 scores of about 0.15 at $t+1$, 0.17 at $t+3$ and 0.35 at $t+5$. Since F1 score reflects the combined balance of precision and recall. These results show that the Static model performs better on this combined detection measure during drift. This also suggests that the Adaptive model, despite its higher precision, misses a larger proportion of actual bottleneck events. This difference is consistent with the Adaptive model applying a stricter decision threshold during the drift phase than the static model, which makes the Adaptive model more conservative in raising alerts and directly reduces its recall (see FIGURE 4).

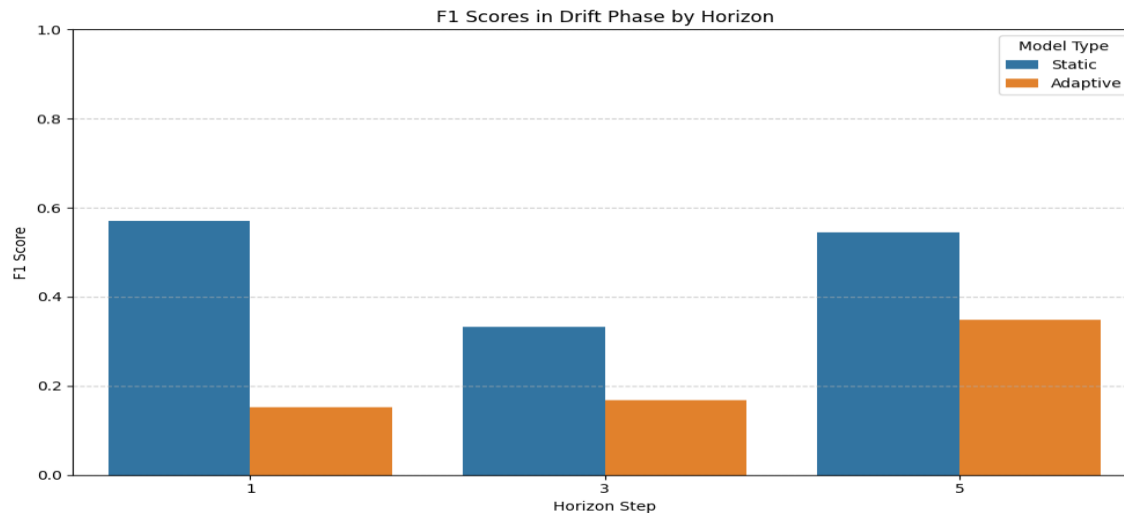


FIGURE 4. F1 score in drift phase

In the drift phase, the balanced accuracy values of the two models are similar across all three forecast horizons (see Figure 5). The Static model has balanced accuracy values of about 0.54 at $t+1$, 0.53 at $t+3$ and 0.55 at $t+5$, while the adaptive model has values of about 0.52 at $t+1$, 0.53 at $t+3$ and 0.56 at $t+5$. These results show that, when class imbalance is considered, the overall performance of the two models is broadly similar, with only seminiiferous across horizons (see FIGURE 4).

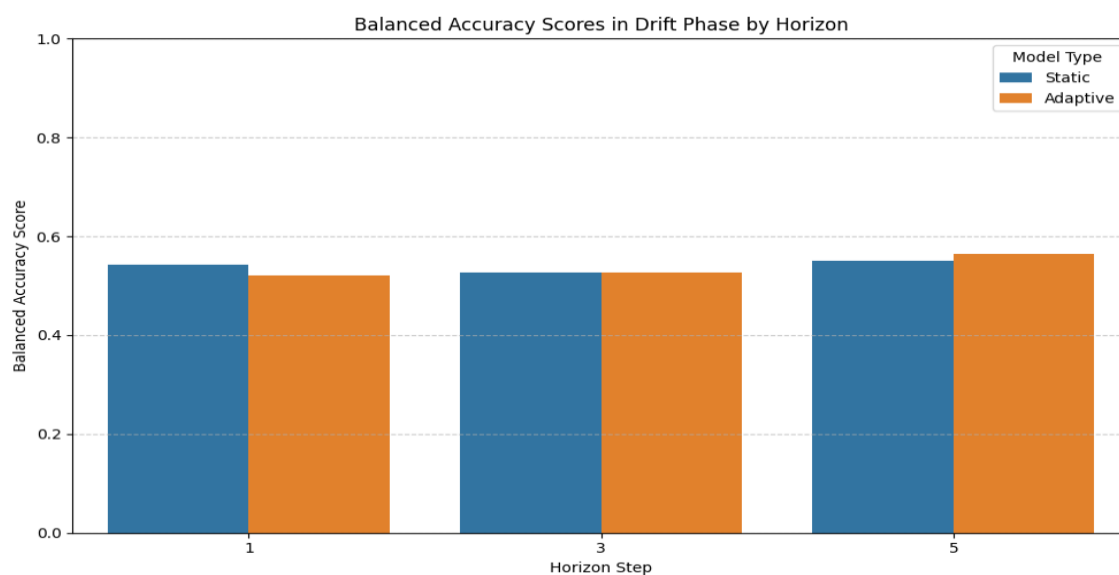


FIGURE 5. Balanced accuracy score

Taken together, the three metrics show that both models can be used for sort term bottleneck forecasting under dynamic operating conditions in the drift phase, but they do not perform in the same way. The Adaptive model performs better in precision, which means its alerts are more often correct. The Static model performs better in F1 score, which indicates stronger combined detection performance and suggests higher recall. In balanced accuracy, the two models show similar overall results. Therefore, the results indicate that the Adaptive and Static models reflect different detection behaviours across $t+1$, $t+3$ and $t+5$, rather than one model being consistently superior on all three-evaluation metrics. Lead time analysis, which measures how early a warning is issued before a bottleneck event, forms a separate part of the evaluation for research question 2 and is reported separately in this thesis (see FIGURE 5).

4.3.3 Comparative Results for Static and Adaptive LSTM

This section compares the performance of the Static LSTM and Adaptive LSTM models for sort term bottleneck forecasting in the drift phase across the forecast horizons $t+1$, $t+3$ and $t+5$ (FIGURE 6). The comparison is based on precision, F1 score and balanced accuracy, as shown in the current figure. Because the values in this figure differ from some earlier charts, the final thesis should use only one confirmed result set consistently throughout all sections. In terms of precision, the results are mixed across horizons. The adaptive model is slightly higher at $t+1$ and $t+5$, with values of about 0.71 compared with about 0.68 for the Static model, while the Static model is higher at $t+3$, with a value of about 0.71 compared with about 0.67 for the Adaptive model. These results indicate that neither model maintains a consistent precision advantage across all forecasting horizons. The adaptive model demonstrates higher precision in two out of the three evaluated cases. The static model outperformed the adaptive model in F1 score across all three forecasting horizons ($t+1$, $t+3$, and $t+5$). The static model recorded scores of 0.52, 0.41, and 0.39, while the adaptive model achieved scores of 0.40, 0.30, and 0.15, respectively.

This shows that adaptive model loses combined detection performance strongly as the prediction horizon becomes longer. In terms of balanced accuracy, the two models remain close across all horizons. The Static model records about 0.53 at $t+1$, 0.54 at $t+3$ and 0.53 at $t+5$, The Adaptive model records about 0.53, 0.51 and 0.51. These values show that neither model has a clear advantage on this metric. At the same time, all balanced accuracy values lie roughly between 0.51 and 0.55, which is only slightly above 0.5. This corresponds to the performance level expected from random classification. This means that, although the two models are similar on balanced accuracy their absolute performance on this metric is limited in the drift phase.

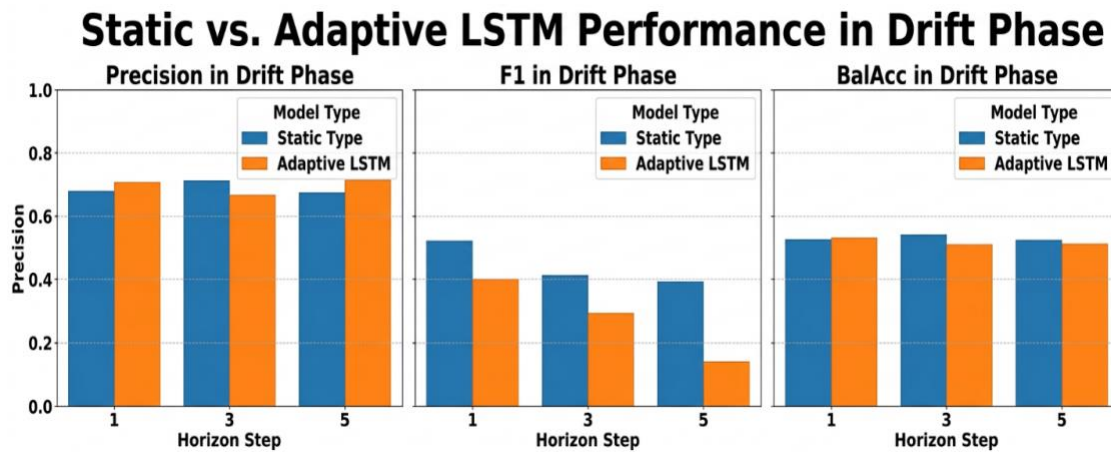


FIGURE 6. Static and adaptive performance

Taken together, the comparative results show that the Static and Adaptive LSTM models exhibit different performance characteristics in the drift phase. Precision is mixed across horizons, with the Adaptive model slightly higher at $t+1$ and $t+5$ and the Static model higher at $t+3$. The Static model achieves higher F1 scores at all horizons and shows a more stable pattern as the forecast horizon increases. Balanced accuracy is similar for both models, but its absolute level remains only marginally above random classification. Therefore, the current chart indicates that the Static model provides stronger overall detection performance in the drift phase, while the Adaptive model shows only a limited precision advantage at some horizons and a clearer decline in combined performance as the forecast horizon increases (see FIGURE 6).

4.3.4 Lead Time analysis

Lead-Time Analysis evaluates how early the model can issue a warning before a bottleneck occurs. The average lead time for the Adaptive model across the three forecast horizons, $t+1$, $t+3$ and $t+5$, is presented in (see FIGURE 7). The results indicate average lead times of approximately 8.6 steps for $t+1$, 7.5 steps for $t+3$ and 7.1 steps for $t+5$. These results show that the Adaptive model can provide advance warnings before the bottleneck event in all three cases, since all lead time values are clearly above zero. This means the model does not only identify bottlenecks but gives warnings several steps before they occur. The pattern in the figure shows that average lead time decreases from $t+1$ to $t+5$. This may appear unexpected, because a longer forecast horizon might normally be assumed to provide more advance notice. However, in this analysis, lead time is not determined directly by the forecast horizon itself. It is measured as the gap between the first qualifying alert within the lookback window and the actual start of the bottleneck event. The reported lead time is determined by the actual timing of model alerts rather than the nominal horizon length. Based on the current results, the interval between the first valid alert and the event boundary narrows as the forecasting horizon increases. It produces the decreasing trend shown in the chart. It is also important to state that this figure represents the adaptive model only. The lead time calculation is based on the adaptive alert logic, including the threshold-based warning rule used during evaluation. Therefore, this chart does not compare Static and Adaptive lead time side by side. On the other side it shows the Early warning behaviour of the Adaptive bottleneck forecasting model across the three horizons.

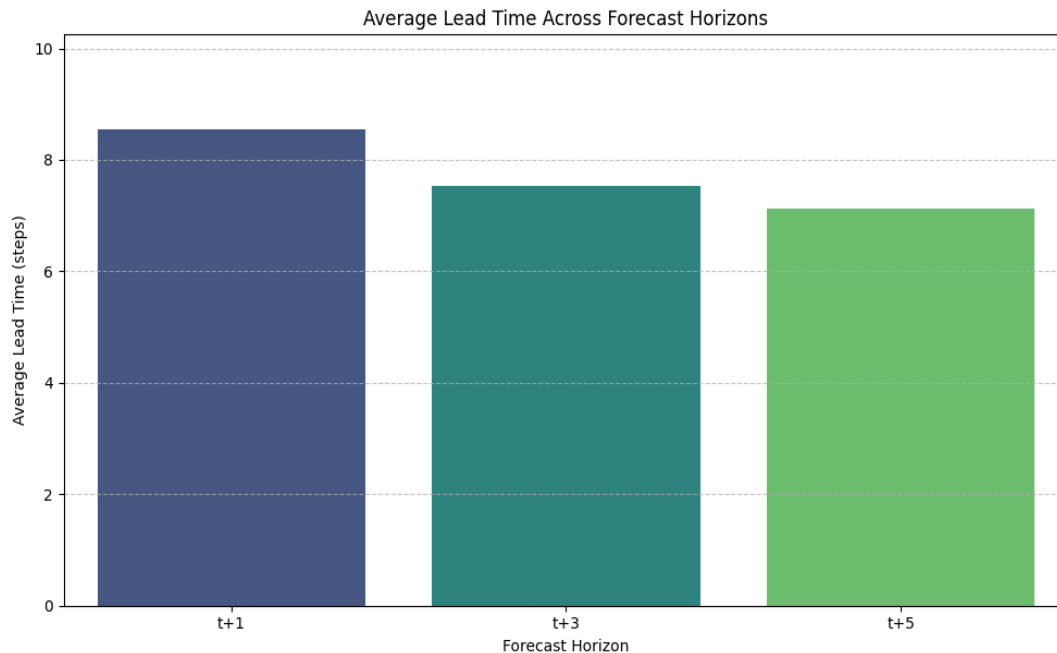


FIGURE 7. Average lead time

The result of this figure is that the Adaptive model provides useful advance warning in all three forecast settings, with the strongest Early warning performance at $t+1$. When this finding is considered together with the earlier precision, F1 score and balanced accuracy results, the overall conclusion for Research Question 2 is that time series machine learning models can forecast sort term bottlenecks under dynamic operating conditions, but their practical value depends not only on detection accuracy but also on how early they can issue a usable warning before the event (see FIGURE 7).

4.4 Drift aware behaviour and KPI analysis

This section examines how the forecasting models behave when operating conditions change and evaluates that behaviour using key performance indicators KPIs. This analysis examines the timeliness and reliability of drift detection, the behaviour of prediction errors within the adaptive cycle time model, and performance fluctuations across the stable, drift, and recovery

phases. Furthermore, it interprets these findings from an operational perspective. The objective is to evaluate the models. It goes beyond simple accuracy, focusing on their stability, responsiveness, and practical utility under nonstationary conditions.

4.4.1 Detection delay and false positives

Detection delay measures the number of steps between the true onset of drift and the first ADWIN detection. False Positives measure the number of drift alarms raised before the true drift begins. In the diagram, the red dashed line marks the true drift onset at step 600, the green triangles mark the ADWIN detection points and the shaded red region marks the main drift phase. The first ADWIN detection appears at approximately step 610, indicating a detection delay of about 10 steps. There are no detection markers before step 600, which means that the number of false positives is zero (see FIGURE 8). The diagram also shows that ADWIN does not trigger only once. It produces multiple detections throughout the drift phase because it continuously monitors the error stream and can signal again when additional distributional changes are detected within its adaptive window. The detection points also continue beyond the drift phase boundary at step 800 and extend into the recovery period. This is consistent with the recovery phase also using modified process conditions rather than returning fully to the original stable baseline. Therefore, the repeated detections after step 800 should not be interpreted as an error in the figure, but as continued response to changing data conditions.

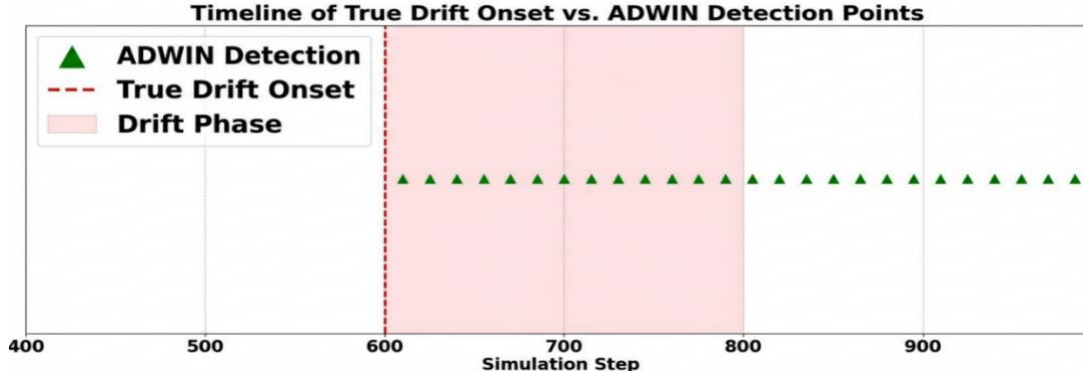


FIGURE 8. Delay and positive detection

The result is that ADWIN detects the start of drift with a delay of approximately 10 steps and produces no false alarms before drift begins. This means the detector is reliable in the stable phase and responsive after the distribution changes. In relation to research question 1, The ADWIN alarm is the event that triggers adaptive retraining in the LSTM pipeline. A detection delay of about 10 steps means the adaptive model receives a retraining trigger soon after drift begins, allowing it to adjust during the drift period rather than remaining fixed throughout it (see FIGURE 8).

4.4.2 Residual analysis for adaptive cycle time prediction

Residual analysis for adaptive cycle time prediction is used to examine the prediction errors of the adaptive model, namely the LSTM+ADWIN model, to assess the stability and bias of its cycle time forecasts under varying operating conditions. In this study, a residual is defined as the difference between the actual cycle time value and the predicted value (Actual – Predicted). A residual close to zero suggests a high level of prediction accuracy. Positive residuals indicate that the model underestimated the actual cycle time, whereas negative residuals indicate that it overestimated it. The distribution of residuals across the three evaluation phases Stable, Drift and recovery is illustrated in the boxplot (see FIGURE 9). For each phase, the box represents the interquartile range, the line within the box denotes the median residual and the whiskers indicate overall spread of the residual values. Accordingly, the figure enables a comparison of both the central tendency and the dispersion of prediction errors across the three phases.

In the stable phase, the residuals are centered close to zero and the spread is relatively smaller, which indicates that the adaptive model has low overall bias and more stable prediction errors under unchanged operating conditions. In the Drift phase, the box and whiskers are clearly wider and the median residual shifts upward to a positive value, which shows that prediction errors become larger and more variable when the process conditions change. The upward shift of the median in the drift phase indicates a systematic underprediction tendency. The model consistently forecasts cycle times that are lower than the actual values, indicating a systematic underestimation bias. During the recovery phase, the distribution of residuals tightens compared to the drift phase, and the median residual shifts back toward zero. These results demonstrate that prediction stability and accuracy improve following the resolution of the primary disturbance. However, the lower whisker in the recovery phase extends further downward than in the stable phase. It suggests that although the central spread is tighter than during drift, occasional larger negative residuals still occur. This means that some overprediction spikes remain present even after the main drift period.

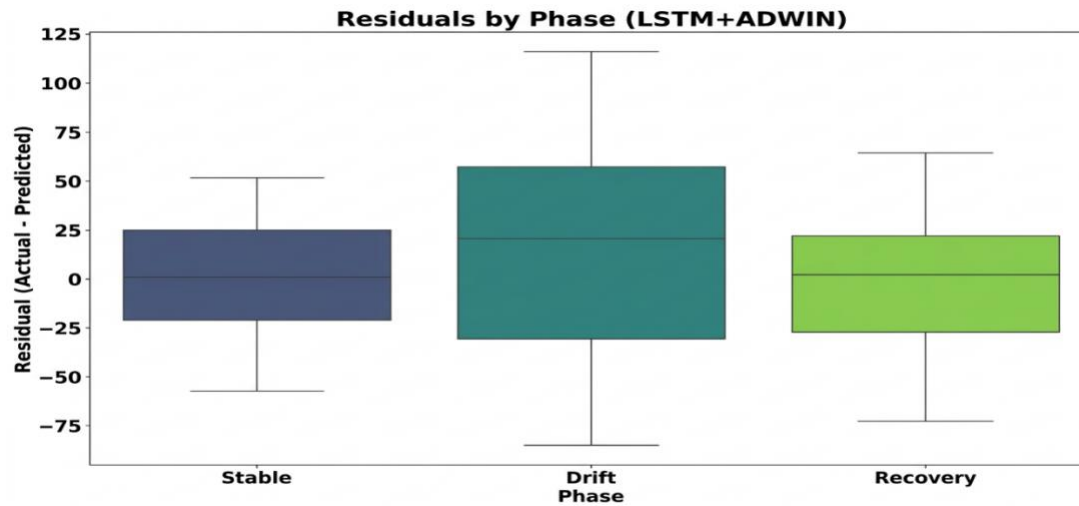


FIGURE 9. Residual in LSTM and ASWIN

The result of this residual analysis is that the adaptive model performs most stably in the stable phase. It shows the largest error magnitude and variability in the drift phase and then partially regains stability in the recovery phase. This pattern is consistent with the residual KPI values, where the mean residual is close to zero in the stable and recovery phases but rises in the drift phase and the standard deviation of residuals is highest during Drift. These findings contribute

directly to research question 1 by showing how the adaptive LSTM+ADWIN. To change operating conditions at the level of individual prediction errors across all three phases (see FIGURE 9).

4.5 Model refinement and trade offs

This section examines how the implemented forecasting models can be improved and analyses the key trade-offs. It was observed during evaluation under dynamic operating conditions. The results demonstrate that both cycle time prediction and bottleneck forecasting benefit from adaptive learning mechanisms. The models can adapt to changing production behaviour. They also reveal that performance improvements are not uniform across all metrics and phases. In particular, the adaptive mechanisms introduce differences in accuracy, stability, responsiveness and alert behaviour that is carefully balanced for practical deployment. This section therefore focuses on refining the adaptive updating logic, evaluating the effect of decision thresholds in bottleneck forecasting and analysing the trade-offs between accuracy and stability, as well as responsiveness and false alarms. The aim is to provide a clearer understanding of how model design choices influence performance and how these choices can be adjusted to better match real industrial requirements.

4.5.1 Refinement of adaptive updating logic

The adaptive updating logic implemented in this study differs between the two forecasting pipelines, as described in Chapter 3. In the cycle time pipeline, retraining is triggered by ADWIN drift detection applied to the squared prediction error stream. In the bottleneck pipeline, retraining is triggered when the recent F1 score falls below 0.65 within the drift interval, subject to a minimum gap of 40 steps between updates, with each retraining carried out on the most recent 70step data window. The results indicate that the ADWIN-based trigger in the cycle time pipeline responded within approximately 10 steps of the true drift onset, with no false positives observed before the drift phase, which indicates that the detection mechanism responded promptly and without false alarms under the simulated conditions. However, the

adaptive LSTM did not consistently outperform the standalone LSTM and ARIMA across all phases and error measures, which suggests that responsive detection alone may not be sufficient if the retraining configuration is not well matched to the rate and magnitude of the drift. In the bottleneck pipeline, the fixed 40step minimum gap and the 70step retraining window were defined without systematic tuning and may not be optimal under all drift conditions. The observed decline in the Adaptive model's F1 score across longer horizons suggests that the current adaptive design may not maintain the same level of detection performance as forecast distance increases. A refinement of this logic could therefore involve adapting the retraining gap or testing alternative retraining window sizes to identify a more robust local update strategy. For example, a dynamic gap that shortens when performance degrades rapidly, or a window size sensitivity test comparing 50, 70 and 100 steps, could help identify a more robust update strategy. Therefore, refinement of the adaptive updating logic in this study refers to making the update decision more closely matched to the observed change in production conditions by adjusting the trigger sensitivity, retraining frequency, or the size and composition of the data window used for each update. Threshold refinement was applied to improve the performance of the bottleneck forecasting mode

4.5.2 Threshold refinement in bottleneck forecasting

In the implemented bottleneck forecasting pipeline, predicted probabilities were converted into binary alerts. It used a fixed threshold of 0.50 for the static model. The phase depends on thresholds of 0.70, 0.55 and 0.45 for the adaptive model, the stable, drift and recovery phases respectively. However, the static model achieved higher F1 scores at all three horizons. It shows that the higher precision of the adaptive model was accompanied by weaker combined detection performance. This pattern is consistent with the stricter drift phase threshold of 0.55 used by the adaptive model. It makes the model more conservative in raising alerts and therefore reduces false positives at the cost of lower recall. In the recovery phase, the lower threshold of 0.45. It was intended to increase alert sensitivity as conditions began to stabilise. This made an adaptive model less conservative than in the drift phase. This shift reflected a strategic attempt to recapture bottleneck events that were previously missed, as the operating

environment. It transitioned away from the highly volatile drift condition and toward a more stable state. The phase dependent threshold design therefore did not improve all bottleneck metrics simultaneously. Instead, the design shifted model behaviours toward varying balances between alert reliability and alert sensitivity across different operating phases. A refinement of this threshold logic would require systematic calibration rather than manual selection. This approach suggests that threshold selection based on systematic calibration is preferable to manual assignment. Specifically, thresholds can be optimized by testing various values on a validation window. It uses a precision recall curve or by aligning them with the operational costs associated with missed bottlenecks (false negatives) versus false alarms (false positives).

4.5.3 Accuracy versus stability

In this study, accuracy is defined as the magnitude of prediction error within a specific phase. The stability refers to the consistency of model performance as the process transitions through the stable, drift, and recovery phases. The cycle time prediction results show that these two properties were not fully aligned. In the stable phase, the standalone LSTM and the adaptive LSTM produced identical results. They both performed slightly better than ARIMA. They indicate that the adaptive mechanism did not provide an additional advantage under unchanged operating conditions. In the drift phase, the results were mixed, ARIMA achieved the lowest MAE and RMSE. The standalone LSTM achieved the lowest MAPE, and the adaptive LSTM did not produce the lowest value on any of the three error measures. In the recovery phase, the adaptive LSTM showed the strongest overall performance, achieving the best RMSE and MAPE values. The standalone LSTM achieved a slightly lower MAE.

These results indicate that no single model was the most accurate in every phase. Instead, model effectiveness changed with the operating condition and with the metric being considered. In this sense, the adaptive LSTM showed greater stability than ARIMA. It produced the strongest overall recovery phase performance after the process had changed. It was not the best model on every error measure in every phase. A similar trade-off is visible in bottleneck forecasting during the drift phase. The Adaptive model achieved higher precision across all three forecast horizons, with values of about 0.79, 0.80 and 0.80 at $t+1$, $t+3$ and $t+5$

respectively, compared with about 0.69, 0.70 and 0.69 for the Static model. However, the Static model achieved higher F1 scores at all three horizons, with about 0.57, 0.33 and 0.54, compared with about 0.15, 0.17 and 0.35 for the Adaptive model. This shows that the Adaptive model produced more reliable alerts in terms of precision, while the Static model achieved better combined detection performance. In terms of stability within the drift phase horizon comparison, the adaptive model maintained nearly constant precision across $t+1$ to $t+5$. The F1 scores remained lower than those of the Static model across all three horizons. Overall, the results show that maximising accuracy or alert correctness in one respect did not guarantee the most stable or strongest overall performance.

4.5.4 Responsiveness versus false alarms

In this study, responsiveness refers to how quickly the ADWIN based mechanism reacts after the onset of drift. The false alarms refer to drift detections raised before any true process change has occurred. These two properties represent a trade-off, since a highly sensitive detector may respond earlier but may also generate unnecessary alarms. A more conservative detector may reduce false alarms, but this may come at the cost of a slower response. The results show that the first ADWIN detection occurred at approximately step 610, while the simulated drift began at step 600. It corresponds to a detection delay of about 10 steps. At the same time, no ADWIN detections were observed before step 600. The false positive count was zero. This indicates that the detector achieved a favourable balance between response speed and alarm reliability. The figure also shows that ADWIN did not trigger only once. Multiple detections occurred throughout the drift phase and continued into the recovery period. This repeated signalling is consistent with the design of the detector, which continuously monitors the prediction error stream. It can react again when additional distributional changes are observed within its adaptive window. Because the recovery phase did not fully return to the original baseline, the system continued to exhibit altered behavior. The detections after step 800 interpreted as continued response to changed process conditions rather than as spurious alarms. In operational terms, this responsiveness is important because each valid ADWIN signal can trigger retraining of the adaptive LSTM. A detection delay of about 10 steps therefore means that the model

receives an update signal. It is relatively early in the drift period, rather than remaining fixed throughout the disturbance. For the purposes of this study, the observed balance was satisfactory. The detector responded soon after the onset of simulated drift and avoided unnecessary alarms during the stable phase. It continued to respond as the process remained in a changed state during the drift and recovery phases.

4.6 Practical challenges and benefits revisited

Building on the practical challenges and benefits identified in Chapter 3, the evaluation results confirm both the operational value and the deployment limitations of drift aware forecasting models in an assembly line environment. One key benefit observed in the results is the ability of the adaptive models to keep performance under changing conditions. In the cycle time prediction task, the adaptive LSTM achieved stronger overall performance than the non-adaptive models in the recovery phase on two of the three error measures, indicating that retraining based on detected change allowed the model to adjust after the disturbed operating regime. In the bottleneck forecasting task, the adaptive model produced higher precision across all forecast horizons. It means that a larger proportion of its bottleneck alerts were correct and operationally reliable.

The lead time analysis showed that the adaptive bottleneck forecasting model was able to detect warnings steps. The model identified bottleneck events across all three forecast horizons. It offers enough lead time for teams to intervene effectively. These results support the practical usefulness of adaptive learning for maintaining prediction quality. It improves alert reliability and enabling earlier response in dynamic production environments. However, several practical challenges are also apparent based on results. First, the adaptive updating mechanism relies on retraining triggered by drift detection or performance monitoring. The ADWIN detector responded with a delay of approximately 10 steps. It produced no false positives and retraining introduces computational overhead. It needs to be assessed against the available processing capacity in a real assembly line. Second, in the bottleneck forecasting pipeline, the retraining logic is based on fixed parameters, including a 40step minimum gap and 70 steps retraining window. The observed decline in the Adaptive model's F1 score at

longer forecast horizons suggests that adaptation may not fully compensate for the increasing uncertainty associated with longer-term predictions. This fixed configuration provides insufficient adaptation under all conditions. Third, Phase dependent decision thresholds add an additional operational requirement. The selecting appropriate thresholds dynamically require additional logic or monitoring. Another challenge relates to the balance between responsiveness and stability. Although the adaptive system detected drift without false alarms and responded within a short delay. It remained difficult to maintain consistent performance across all metrics and phases. The results show that improvements in one metric, such as precision, may come at the expense of other metrics, such as recall or F1 score. They were sometimes performed by reductions in other metrics. For example, F1 score, reflecting inherent trade-offs in model design and configuration. Overall, the results show that adaptive forecasting provides clear operational benefits in dynamic environments. It maintains in performance after process changes, improving the reliability of bottleneck alerts and providing earlier warning. At the same time, these benefits depend on carefully designed, threshold selection and retraining strategies.

4.6.1 Challenges under simulated drift

Simulated drift control changes rather than drift observed in production environments. In the bottleneck forecasting implementation, drift was introduced by multiplying `Processing_Time` by 1.8. The `Machine_Availability` by 0.4 during the drift phase, which represents a relatively operating change. In the real industrial drift often consists of smaller, gradual, or mixed shifts that are more challenging to detect and handle. This creates a first challenge for interpretation. The reported adaptive behaviour may not generalise directly to real production settings. A second challenge is that the drift evaluation window covers only 200 steps, corresponding to records 600 – 799. This limits the number of bottleneck events available for metric calculation within that phase. It may contribute to variability in the reported drift phase results. A third challenge arises from the recalculation of bottleneck labels from the simulated `Processing_Time` and `Machine_Availability` series. It changes the class distribution across phases and introduces class imbalance. For this reason, standard accuracy is not sufficient as a primary evaluation measure and greater emphasis is placed on balanced accuracy and F1 score.

A fourth challenge is that the adaptive bottleneck model relies on phase dependent decision thresholds. They are feasible in simulation because the phase boundaries are predefined but would require additional logic in real deployment. The operating phase is not known in advance. Together, these challenges mean that the reported results are interpreted as evidence of model behaviour under controlled simulated conditions.

4.6.2 Benefits of adaptive forecasting and early warning

In this thesis, the benefits of adaptive forecasting and early warning refer to the practical advantages created by the implemented drift aware forecasting system in the assembly line environment. The first benefit is that the forecasting models do not remain fixed when production conditions change. In the cycle time pipeline, ADWIN monitors the prediction-error stream and triggers retraining of the adaptive LSTM when drift is detected, while in the bottleneck pipeline retraining is triggered when the recent F1 score falls below the defined threshold during the drift interval. This allows the system to adjust to changing operating conditions instead of continuing with an outdated model.

The second benefit is early warning capability. The bottleneck forecasting model produces predictions at $t+1$, $t+3$ and $t+5$ and the lead-time analysis shows that warnings were issued on average approximately 7 to 9 steps before a bottleneck event occurred, giving operators time to intervene before throughput is disrupted. The third benefit is improved reliability of bottleneck alerts. The adaptive model achieved higher precision than the static model during the drift phase. The values of approximately 0.79 and 0.80 compared with 0.69 and 0.70, meaning that a larger share of its alerts was correct. The adaptive model achieved higher precision at the cost of lower F1 scores. In operational settings, false alarms reduce operator trust and trigger unnecessary interventions. The fourth benefit is phase-aware alert sensitivity, as the implementation employs phase-dependent decision thresholds. It allows the system to change alert sensitivity across stable, drift and recovery conditions rather than relying on a single fixed alert rule throughout. The framework also produces operationally usable outputs, including predicted cycle times, predicted bottleneck events, drift alarms and KPI summaries.

It supports monitoring and decision making in the production environment. The implemented adaptive forecasting design maintains performance under changing conditions. It also provides earlier bottleneck warnings. It produces reliable alerts and supports continuous monitoring. Both contribute to practical operational response in a dynamic assembly line setting.

4.6.3 Deployment oriented considerations

In this thesis, deployment oriented considerations refer to the practical requirements and constraints. It is addressed when moving the implemented forecasting system from controlled simulation into a real assembly line. Both forecasting pipelines rely on sequential input streams arriving and require ongoing evaluation of model validity as new data arrives. The system supports uninterrupted data flow, real time preprocessing and repeated prediction cycles. Another consideration is drift detection and model updating in real time. In the cycle time pipeline, ADWIN monitors prediction error and triggers retraining when drift is detected. In the bottleneck pipeline retraining is triggered by performance degradation measured through recent F1 score.

In a deployment setting, each retraining step requires fitting an LSTM model on recent data windows. A further consideration is threshold selection and decision logic. The bottleneck forecasting system uses phase dependent thresholds of 0.70, 0.55 and 0.45. In real deployment the current operating phase is not known in advance. This means that an additional mechanism is required either to detect the current phase or to adjust thresholds dynamically. Another requirement is scaler validity under changing data distributions. In the cycle time pipeline, scaling is recalibrated during retraining. In the bottleneck pipeline a fixed scaler is used. Under real drift conditions, feature distributions may shift, and this can affect prediction reliability if scaling is not updated appropriately. The system also ensures low false alarm behaviour and operational reliability. Excessive false drift detections or bottleneck alerts reduce trust and may lead to unnecessary interventions. The balance between responsiveness and false alarms is maintained in deployment. In addition, computational cost and retraining frequency are important constraints. The bottleneck pipeline includes a minimum 40 step gap between

retraining events and each update involves training horizon specific models. In a real production system, this retraining schedule aligns with available processing capacity and the speed of process change. Finally, deployment requires that the system produces operationally interpretable outputs. It predicted cycle times, bottleneck alerts, drift signal and KPI summaries. The production operators can understand and act on the results. Overall, deployment oriented considerations in this thesis highlight. It needs for continuous monitoring, real time adaptation, scalable retraining, appropriate threshold management and reliable, interpretable outputs.

4.7 Limitations and generalizability

This section discusses the main restrictions of the study. It also examines the extent to which the findings can be generalised beyond the specific experimental setting. In this thesis, the limitations arise mainly from the use of a single manufacturing dataset. The controlled simulation of drift conditions, and fixed methodological choices related to model updating and threshold selection. The design choices in model updating and threshold selection significantly influence model performance under dynamic conditions. The generalizability discussion then considers the implemented forecasting framework. The framework is applicable to other assembly line environments. These environments have similar data characteristics, operating conditions, and forecasting requirements.

4.7.1 Dataset and simulation limitations

This study is subject to several limitations related to the dataset and the simulation design. First, the analysis is based on a single manufacturing dataset containing 1,000 observations. It consists of 13 variables, of which 995 records remained after cleaning. Although this dataset is suitable for implementing and testing the proposed forecasting framework, further validation on additional datasets is required. The use of a single dataset limits the diversity of production conditions represented in the study. As a result, the findings reflect the characteristics of this

specific dataset rather than the full range of variability. The use of a single dataset limits the diversity of production conditions that occur across different assembly line environments. Second, the original dataset does not contain explicit natural concept drift. For this reason, drift was introduced experimentally through controlled changes in the target related variables. In the cycle time prediction pipeline, `Processing_Time` was multiplied by 1.6 during the drift phase and by 1.15 during the recovery phase. In the bottleneck forecasting pipeline, `Processing_Time` was multiplied by 1.8 and `Machine_Availability` by 0.4 during the drift phase. The `Processing_Time` was multiplied by 1.3 during recovery. These settings provide a clear experimental structure for evaluating adaptive behaviour. They represent simulated rather than naturally observed production changes. Real industrial drift is typically smaller in magnitude, more gradual, and less predictable; consequently, it may be more challenging to detect and adapt to. Third, the drift evaluation window is limited. In the bottleneck forecasting task, the drift phase covers records 600 – 799, corresponding to 200 steps. This relatively short interval limits the number of bottlenecks. It is available for metric calculation within that phase and may contribute to variability in the reported drift phase results. Fourth, in the bottleneck forecasting implementation, the target variable is recalculated after simulation from the modified `Processing_Time` and `Machine_Availability` series. This changes the class distribution across phases and introduces class imbalance as part of the simulated drift scenario. Consequently, the reported results should be interpreted in the context of this constructed target behaviour rather than as direct evidence from naturally occurring bottleneck events. Overall, these dataset and simulation limitations mean that the reported findings should be interpreted as evidence of model behaviour under controlled experimental conditions, not as direct proof of performance in all real industrial assembly line environment.

4.7.2 Methodological limitations

This study also has several limitations related to the methodological design and implementation of the forecasting framework. First, the adaptive updating logic is based on predefined rules rather than systematic optimisation. In the cycle time prediction pipeline,

retraining is triggered by ADWIN based on changes in squared prediction error, while in the bottleneck forecasting pipeline retraining is triggered when the recent F1 score falls below a fixed threshold within the drift interval, subject to a minimum gap of 40 steps and a fixed 70step retraining window. These parameters were selected for implementation rather than tuned through systematic validation and different configurations could lead to different adaptation behaviour. Second, the bottleneck forecasting approach relies on phase dependent decision thresholds (0.70, 0.55 and 0.45) to convert predicted probabilities into binary alerts. While this allows controlled evaluation of model behaviour across phases, it assumes that the operating phase is known in advance. In a real deployment setting, the current phase would not be explicitly labelled, and additional logic would be required to determine appropriate thresholds dynamically. Third, the modelling approach is limited to specific model classes and update strategies. For cycle time prediction, only ARIMA, standalone LSTM and ADWIN-based adaptive LSTM are evaluated and for bottleneck forecasting only static and adaptive LSTM models are used. Alternative models or adaptation strategies, such as incremental learning or alternative drift detection methods, are not considered in this study. Fourth, the evaluation design is based on a fixed sequence structure and predefined forecast horizons. All models use a 10step input window and bottleneck forecasting is limited to $t+1$, $t+3$ and $t+5$ horizons. While this provides a consistent comparison framework, it restricts the exploration of how different window sizes or longer forecasting horizons may affect performance. Fifth, the normalisation strategy differs between the two pipelines. In the cycle time pipeline, scaling is recalibrated during retraining. In the bottleneck pipeline a fixed scaler fitted on the initial training data is used throughout evaluation. Under changing data distributions, this fixed scaling approach not fully reflect the current feature distribution and may affect prediction quality. Overall, these methodological limitations indicate that the results are dependent on several items. Examples include the chosen model structures, parameter settings, and evaluation design. The findings therefore reflect the behaviour of the implemented framework rather than the full range of possible adaptive.

4.7.3 Generalizability to similar assembly line settings

The generalizability of this study refers to the extent to which the implemented forecasting framework can be applied to other industrial assembly line environments with similar characteristics. The framework is designed for production systems that generate sequential data and include variables such as processing time, machine availability, energy consumption, upstream flow behaviour and machine or station identifiers. It is therefore most applicable to assembly line settings where production behaviour evolves over time, bottlenecks emerge from changing operating conditions and continuous data monitoring is available. The core components of the framework sort term cycle time prediction, bottleneck forecasting, drift aware adaptation and Early warning generation are not specific to a single production line and can be transferred conceptually to similar manufacturing environments that meet these conditions. In particular, the use of sequence-based models, sliding-window inputs and adaptive retraining mechanisms is aligned with general requirements for forecasting in non-stationary industrial processes. However, the generalizability of the reported results is limited by the controlled experimental setting used in this study. The evaluation is based on a single dataset and on simulated drift conditionate forecasting framework is tested under predefined stable, drift and recovery phases. In addition, the bottleneck target is constructed from threshold-based rules applied to simulated variables and the class distribution changes across phases as part of the experimental design.

The reported performance reflects behaviour under controlled experimental conditions. It does not represent direct evidence of performance in a live industrial environment. The framework is reasonably generalizable to assembly line systems with similar data structures, comparable sources of operational variability, and similar forecasting objectives. These objectives include short term cycle time prediction and bottleneck warning. Application to different industrial contexts requires additional validation. This applies to systems with different process dynamics, data availability or bottleneck definitions. In such cases, the model structure, retraining logic, and threshold settings may need to be adapted.

5 DISCUSSION

This chapter interprets the results presented in chapter 4. It is related to the three research questions and assesses their implications for forecasting in dynamic assembly line environments. The discussion focuses on how drift aware and nonadaptive time series machine learning. Models perform under stable, drift and recovery conditions and what these results indicate about their practical usefulness. Each research question is considered separately, covering cycle time prediction, bottleneck forecasting and real world deployment considerations. The chapter also examines the strengths and limitations of the preformed framework and highlights key trade offs. A trade-off is observed between accuracy, robustness, responsiveness, and operational reliability. The aim is to provide a clear interpretation of the findings and connections the experimental results.

5.1 Interpretation of research question 1

The results show that integrating ADWIN drift detection with LSTM improved the robustness of short term cycle time prediction under nonstationary conditions. The benefit was observed in the recovery phase, while the drift phase showed mixed results across error measures and the stable phase. It showed no difference between the adaptive and non-adaptive LSTM models. research question 1 asks how the integration of concept drift detection techniques, such as ADWIN, with time-series machine learning models enables effective adaptation to concept drift while maintaining forecasting accuracy. It includes an LSTM, which influences the model accuracy and robustness of short-term cycle time prediction in assembly lines when compared with standalone LSTM and ARIMA. In this study, robustness is understood as the ability to maintain acceptable performance when operating conditions change across the stable, drift and recovery phases. In the stable phase, the standalone LSTM and the adaptive LSTM produced identical results. This shows that, under unchanged operating conditions, the ADWIN based adaptive mechanism did not provide an additional accuracy benefit over the

nonadaptive LSTM. In this phase, no major process change was present, so adaptive updating was not required. In the drift phase, the results were mixed across the three error measures. ARIMA achieved the lowest MAE and RMSE, while the standalone LSTM achieved the lowest MAPE. The adaptive LSTM did not produce the lowest value on any of the three error measures in this phase. This means that, under the specific simulated drift conditions used in this study, integrating ADWIN did not provide a clear accuracy advantage over the non-adaptive alternatives during the drift phase itself. In the recovery phase, the adaptive LSTM showed the strongest overall performance. It achieved the best RMSE and MAPE values, while the standalone LSTM achieved a slightly lower MAE. This result indicates that the adaptive updating mechanism became more beneficial after the process had changed and moved into a partially recovered condition. In this sense, the adaptive model showed greater robustness than ARIMA and stronger overall recovery-phase performance than the non-adaptive LSTM.

When the results are interpreted across all three phases, they show that no single model was best on every metric in every phase. The stable phase showed no benefit from adaptation, the drift phase produced mixed results, and the recovery phase showed the clearest benefit of the adaptive LSTM. This pattern indicates that the influence of ADWIN integration was phase-dependent rather than uniform. Its main contribution was not a consistent reduction of all error measures in every condition, but improved ability to maintain forecasting performance after the production process had shifted away from the original operating regime. The drift detection KPIs support this interpretation. The first ADWIN detection occurred at approximately step 610, while the simulated drift began at step 600, corresponding to a detection delay of about 10 steps. No false positives were observed before the start of drift. Multiple detections also occurred during the drift and recovery periods, indicating continued response to changed operating conditions.

These results show that the adaptive mechanism responded reliably to the simulated change and provided update signals without unnecessary alarms during the stable phase. Overall, the answer to research question 1 is that integrating ADWIN with LSTM did not improve cycle time prediction accuracy uniformly across all phases. It did improve robustness under changing conditions. The strongest evidence of this benefit appears in the recovery phase where the adaptive LSTM showed the best overall performance. The drift phase remained mixed, and the

stable phase showed no adaptive advantage. Compared with ARIMA, the adaptive LSTM was better able to maintain useful performance after the operating condition had changed, which makes the drift aware approach more suitable for dynamic assembly line forecasting.

5.2 Interpretation of research question 2

The results show that time series machine learning models can be used to forecast sort term bottlenecks in an assembly line under dynamic operating conditions. The extend of usefulness depends on the evaluation metric and on how early the model can issue a warning. It occurs before the event. In this study, bottleneck forecasting was implemented as a sort of term classification task using sequential production data and the models. They were evaluated under changing operating conditions across multiple forecast horizons. The framework was designed specifically for dynamic assembly line settings. Its emphasis is on early warning capability, continuous monitoring, and phase-based evaluation. The results indicate that both the static and adaptive LSTM models were able to forecast bottleneck events in the drift phase. They did not perform in the same way. Across the reported result sections, the common conclusion is that the two models showed different detection behaviours.

One model was uniformly superior on all measures. The adaptive model showed an advantage in precision in the result interpretation sections, meaning that a larger proportion of its alerts were correct. The static model achieved higher F1 scores and therefore stronger combined detection performance. Balanced accuracy remained similar for the two models and stayed only slightly above random classification. This means that the models could forecast sort term bottlenecks. Their effectiveness depended on whether the emphasis was placed on alert reliability or on broader detection performance. The lead time results are central to answering research question 2. The Adaptive model provided positive lead time at all three forecast horizons, with average values of approximately 8.6 steps at $t+1$, 7.5 steps at $t+3$ and 7.1 steps at $t+5$. The model issued warnings several steps before bottleneck events occurred. It was not limited to identifying bottlenecks at the moment of occurrence. It provided advance warnings that support intervention before throughput is disrupted. The strongest early warning

performance was observed at $t+1$. Forecasting short term bottlenecks under dynamic conditions involves a trade off between early warning and detection quality. The adaptive model provided useful early warnings across all three horizons. However, combined detection performance weakened as the forecast horizon increased. The static model showed a more stable F1 score pattern across horizons. The adaptive model showed a stronger decline in combined performance as forecast distance increased. Time series machine learning models can forecast short term bottlenecks. However, practical value becomes more sensitive to model design and threshold logic as the prediction horizon increases. Regarding research question 2, both models can forecast short term bottlenecks to a meaningful extent. The goal is to provide usable early warnings before bottlenecks occur. Forecasting quality is not uniform across all metrics. The adaptive model provides more reliable alerts in terms of precision and lead time. The static model provides stronger combined detection performance in terms of F1 score. The practical usefulness of these models depends not only on bottlenecks detection, but also on how detection accuracy, alert reliability, and advance warning are balanced.

5.3 Interpretation of research question 3

The results indicate that deploying drift aware cycle time and bottleneck forecasting models requires more than predictive accuracy alone. The practical requirements include continuous data monitoring, repeated reassessment of model validity, reliable drift handling, operationally interpretable outputs and retraining process that can function within the timing and resource limits of production use. In this study, these requirements were reflected in the implementation through sequential data handling, phase based KPI reporting, adaptive updating logic and operational outputs such as predicted cycle times, bottleneck alerts, drift alarms and performance summaries. The main practical challenge identified in the study is that the forecasting framework was evaluated under controlled simulated drift rather than naturally observed industrial change. In the cycle time pipeline, drift was introduced by modifying `Processing_Time` and in the bottleneck pipeline by modifying both `Processing_Time` and `Machine_Availability`. This made it possible to test adaptive behaviour systematically, but it also means that real industrial environments may produce more gradual, irregular, or mixed changes than those represented in the experiment. As a result, drift detection and retraining

behaviour in real-world deployments may be less clearly defined than in the simulated setting. A second challenge is the design of the adaptive update mechanism itself. In the cycle time pipeline, ADWIN monitors squared prediction error and triggers retraining when drift is detected. In the bottleneck pipeline, retraining depends on recent F1 score, a fixed threshold, a 40 step minimum gap 70 step retraining window. These choices enabled the implementation of adaptive behaviour, but they were fixed design choices rather than fully optimized deployment settings. This means that successful real world use would require careful adjustment of retraining frequency, data window size and trigger sensitivity to the actual speed. Another practical challenge concerns threshold management in bottleneck forecasting. The adaptive bottleneck model uses phase dependent decision thresholds of 0.70, 0.55 and 0.45 for stable, drift and recovery conditions. This is effective for controlled evaluation, but in a real assembly line environment the current operating phase labelled. Practical deployment therefore requires an additional mechanism to identify the current condition or adjust thresholds dynamically. The study also highlights implementation constraints related to scaling and computational cost. In the cycle time pipeline, scaling is recalibrated during retraining.

In the bottleneck pipeline, a fixed scaler is reused throughout evaluation. Under real drift conditions, feature distributions may shift. This can reduce prediction quality if scaling is not updated. Retraining requires computational resources. This is especially the case in the bottleneck pipeline, where separate horizon specific models are maintained. Deployment therefore requires sufficient processing capacity. Retraining schedules align with the speed of production change. Despite these challenges, the study identifies clear practical benefits. The adaptive models maintained useful performance under changing conditions. In bottleneck forecasting, the adaptive model provided positive lead time across all three horizons. Warnings were issued several steps before bottleneck events occurred. The adaptive model also achieved higher precision during drift. A larger share of its alerts was correct. These results show that drift aware forecasting supports earlier intervention. It also provides more reliable warning behaviour and better responsiveness to changing conditions compared to non adaptive forecasting. Regarding research question 3, deploying drift aware forecasting models in the industrial assembly line environments is practically feasible. The system supports continuous monitoring, adaptive retraining, reliable threshold management, and interpretable outputs. The

benefits include earlier warnings, improved responsiveness, and more reliable alerts under nonstationary conditions. These benefits depend on careful management of retraining logic, computational cost, scaling validity, threshold selection, and the differences between simulated and real drift conditions.

5.4 Discussion summary

The discussion shows that the value of drift aware forecasting in this study was not identical across all tasks, phases, or evaluation measures. For cycle time prediction, integrating ADWIN with LSTM did not improve performance under stable conditions. They did not produce the best result on every error measure during drift, but it showed the clearest benefit in the recovery phase. The adaptive model achieved the strongest overall performance. This indicates that the main contribution of drift aware adaptation was improved robustness after operating conditions had changed. For bottleneck forecasting, the results show that time series machine learning models were able to provide short term warnings under dynamic conditions, with positive lead time. However, the adaptive and static models did not dominate on the same metrics. The adaptive model achieved higher precision and more reliable alerts.

The static model achieved better combined detection performance in terms of F1 score. The discussion also shows that practical deployment depends on predictive performance. Effective deployment of drift aware forecasting in real assembly line environments requires several conditions. These include continuous monitoring, adaptive updating logic, threshold management, computational feasibility, and operationally interpretable outputs. Overall, the study demonstrates that drift aware time series machine learning can support short term prediction and early warning in dynamic production settings. Its practical value depends on the balance between accuracy, robustness, and responsiveness.

REFERENCES

- Alenezy, Abdullah H., Mohd Tahir Ismail, Jamil J. Jaber, S. Al Wadi, and Rami S. Alkhawaldeh. 2022. "Hybrid Fuzzy Inference Rules of Descent Method and Wavelet Function for Volatility Forecasting" edited by A. Kumar. *PLOS ONE* 17(12):e0278835. doi:10.1371/journal.pone.0278835.
- Bifet, Albert, and Ricard Gavaldà. 2007. "Learning from Time-Changing Data with Adaptive Windowing." Pp. 443–48 in *Proceedings of the 2007 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics.
- Dallery, Yves, and Stanley B. Gershwin. 1992. "Manufacturing Flow Line Systems: A Review of Models and Analytical Results." *Queueing Systems* 12(1–2):3–94. doi:10.1007/BF01158636.
- Davis, Jesse, and Mark Goadrich. 2006. "The Relationship between Precision-Recall and ROC Curves." Pp. 233–40 in *Proceedings of the 23rd international conference on Machine learning - ICML '06*. Pittsburgh, Pennsylvania: ACM Press.
- Gomes, Heitor M., Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfahringer, Geoff Holmes, and Talel Abdesslem. 2017. "Adaptive Random Forests for Evolving Data Stream Classification." *Machine Learning* 106(9–10):1469–95. doi:10.1007/s10994-017-5642-8.
- Kashpruk, Nataliia, Cezary Piskor-Ignatowicz, and Jerzy Baranowski. 2023. "Time Series Prediction in Industry 4.0: A Comprehensive Review and Prospects for Future Advancements." *Applied Sciences* 13(22):12374. doi:10.3390/app132212374.
- M. K., Saravana, and Arunalatha J. S. 2025a. "Adaptive MTS Forecasting with LSTM-ADWIN for Concept Drift Detection." *Journal of Intelligent Systems in Current Computer Engineering* 03. doi:10.2174/0130505070403478251003052646.
- M. K., Saravana, and Arunalatha J. S. 2025b. "Adaptive MTS Forecasting with LSTM-ADWIN for Concept Drift Detection." *Journal of Intelligent Systems in Current Computer Engineering* 03. doi:10.2174/0130505070403478251003052646.
- Pagone, Emanuele, Yousef Haddad, Ludovico Barsotti, Gino Dini, and Konstantinos Salonitis. 2023. "A Stochastic Evaluation Framework to Improve the Robustness of Manufacturing Systems." *International Journal of Computer Integrated Manufacturing* 36(7):966–84. doi:10.1080/0951192X.2022.2162592.
- Richard, Rene, and Nabil Belacel. 2024. "A Conditioned Unsupervised Regression Framework Attuned to the Dynamic Nature of Data Streams."

Sun, Yange, Zhihai Wang, Haiyang Liu, Chao Du, and Jidong Yuan. 2016. "Online Ensemble Using Adaptive Windowing for Data Streams with Concept Drift." *International Journal of Distributed Sensor Networks* 12(5):4218973. doi:10.1155/2016/4218973.

Tian, Huixin, Chenning Zhao, Jueping Xie, and Kun Li. 2024. "Dynamic Operation Optimization of Complex Industries Based on a Data-Driven Strategy." *Processes* 12(1):189. doi:10.3390/pr12010189.

Wang, Chien-Chih, Chun-Hua Chien, and Amy J. C. Trappey. 2021. "On the Application of ARIMA and LSTM to Predict Order Demand Based on Short Lead Time and On-Time Delivery Requirements." *Processes* 9(7):1157. doi:10.3390/pr9071157.

Ziya. 2023. Version Version 1 (135.16 kB).
<https://www.kaggle.com/datasets/ziya07/manufacturing-production-data>.